Auto Scheduler Code Guide

A beginner friendly system manual and code reference

19 October 2026

The application turns uploaded class schedules and student assistants' personal schedules into regular duty assignments. It also supports manual assignments, date-specific events, relievers, weekly workload reports, room-based Excel exports, and read-only archives. Most working changes are stored on this device first and sent to Supabase afterward.

Start with Chapter 0 if you have never used the application or read program code. Chapters 1 through 5 explain its foundations. Sections 6A through 6I explain CP SAT and recommendation ranking. Chapters 7 through 15 follow user actions. Chapters 16 through 18E explain persistence and the complete database layout. Chapters 19 through 25 cover desktop operation, testing, worked scenarios, troubleshooting, and system boundaries. The final reference connects these explanations to individual source files and functions. Examples use made-up students and classes.

The important distinction is between the original class schedule, the duty assignments linked to that schedule, and an archive of both. These are separate pieces of data. Removing a duty does not delete a class, and viewing an archive does not replace the active working schedule.

Reading route

What you want to understand	Read first
What each part of the application does	Chapters 1 and 2
A first walkthrough with no technical background	Chapter 0
What data is saved and how CSVs stay separate	Chapters 3 and 16
Uploading and validating schedules	Chapter 4
How CP SAT works and how our solver makes assignments	Chapter 5 then sections 6A through 6I and Chapter 7
Manual duties and relievers	Chapters 8 and 9
Events and assistant availability	Chapters 10 and 11
Calendars and weekly workload rankings	Chapters 12 and 13
Archiving and exporting	Chapters 14 and 15
Supabase and deleted events	Chapters 16 through 18
Every database column and the JSON stored inside it	Sections 18A through 18E

What you want to understand	Read first
What happens in common and unusual situations	Chapters 22 and 23
Checking saved data without deleting anything	Chapter 24
Important boundaries and a demonstration checklist	Chapter 25
Building and closing the desktop app	Chapter 19
Testing and explaining the system in a defense	Chapters 20 and 21
Where a specific file belongs	Source file reference and symbol index

0 Start here if the system is new to you

What problem the application solves

The department already has a class timetable. Student assistants also have their own classes. Someone must decide which assistant can cover each department duty without attending two things at once or exceeding the regular-duty settings. Our application helps make those decisions, records manual changes, manages extra events and temporary replacements, and keeps historical copies.

Creating a schedule assigns assistants to existing class duties. It does not move the department's original classes to different rooms or change the students' enrolled class times. The input timetable supplies those times. The solver chooses who covers eligible work.

The people and records in an example

Anna Santos and Ben Cruz are student assistants. CCS 1001 is a department class in MT102 every Monday from 7 AM to 10 AM. Anna's personal schedule contains Mathematics every Monday from 8 AM to 9 AM. Ben has no personal class then. Anna is unavailable for that whole overlapping duty; Ben may be eligible if the workload and other rules allow it.

A department class is not an assistant profile. A personal class is not a duty. A duty assignment links an assistant to a department class. An event is extra dated work. A reliever temporarily covers an existing duty on a particular date. An archive captures these records together so they can be viewed later.

What the two tabs mean

The Schedule tab organizes the department's work by rooms and dates. It is the place to inspect classes, room conflicts, events, and assignment actions. The Student Assistant tab organizes information by person. Its calendar combines that person's personal classes, regular duties, assigned events, and replacement duties for the selected week.

Switching tabs changes the presentation of saved data. It should not delete assignments or start the solver. Moving to another week changes which dated events and relievers are shown; recurring weekday duties still apply unless date-specific rules such as an exam date suppress them.

A first complete workflow

First, upload the department schedule. The app checks its format and creates a usable timetable. A recognized file can still contain unresolved entries; review TBA and conflict information rather than assuming every imported row is suitable for assignment.

Next, add an assistant and upload that student's personal schedule. Repeat for the other assistants. These profiles belong to the currently uploaded schedule's identifier. Adding a profile alone does not promise that the student will immediately receive duty hours.

Click Create schedule. Scheduling Settings opens. Choose the daily limit, weekly limit, and duty gap. Leave the full-reschedule option unchecked for normal incremental use. Click Proceed. The dialog closes and the button shows Creating schedule while the backend calculates. The moving fill is an activity animation, not a measured percentage.

Review the result in both tabs. Check assistant availability, uncovered duties, and the weekly workload list. You may assign eligible assistants manually to unassigned duties. Those changes are saved as working assignments; you do not need to rerun the solver just to save them.

For an event, add its title, dates, rooms, and times. Assign assistants through the relevant event card. For a temporary duty replacement, use Find Reliever on the existing assigned duty. These are different operations: one creates dated event work, and the other covers work that already has an original assistant.

When the schedule is ready to preserve, save an archive with a unique name, school year, and semester. After a successful local archive save, the current schedule closes. View the archive from Archives when needed. It remains a historical preview rather than becoming editable active work.

What saving means in this application

There are four distinct outcomes. A form may still contain an unsaved draft. A completed action may have been saved on this device. That local change may subsequently have synchronized to Supabase. An archive is an additional historical snapshot. Do not treat these as interchangeable states.

Closing a draft is different from removing saved data. Removing the active CSV is different from deleting the original file on disk. Viewing another week is different from creating another timetable. Exporting Excel is different from backing up every database field. The later chapters explain these distinctions through examples.

A plain language guide to code terms

A variable is a named value, such as the selected date. A function is a reusable operation, such as formatting a time. An object groups named values, such as an assistant's ID and name. An array is a list of records. A map stores values under keys, such as the roster for each uploaded schedule.

A component is a piece of the React interface. State is the data a component currently uses to draw itself. A callback is a function passed to another component so it can report an action. A request sends data to a service; a response brings back the result. An asynchronous operation can finish later, so code must await it or handle its eventual result.

A database table holds rows with named columns. JSON is a way to represent nested objects and lists. PostgreSQL JSONB stores that structure inside one column. A schema describes the database's tables, columns, types, and rules. Sections 18A through 18E explain the actual schema using those definitions.

Sources: `frontend/src/ui/app/App.tsx`, `frontend/src/ui/student-assistants/StudentAssistantPanel.tsx`, `frontend/src/ui/schedule/ScheduleCalendar.tsx`, and `frontend/src/storage/`.

1 The application in one picture

Three working layers

Electron provides the desktop window. Inside that window, React renders the calendar, forms, menus, and dialogs. React runs TypeScript that has been compiled to JavaScript. It does not run the Python solver directly.

The interface sends HTTP requests to Flask at `http://127.0.0.1:5000`. This address refers to the user's own computer. Flask parses schedule rows, runs the OR-Tools solver, validates requests, and talks to Supabase through repository functions.

Supabase stores shared data remotely. Local browser storage and IndexedDB let the interface retain working data and archives when the internet is unavailable. The backend is still needed for importing and solving, even when the internet is disconnected.

```
User clicks a control
        |
React interface inside Electron
        |-- localStorage for working data and retry queues
        |-- IndexedDB for archive snapshots
        |
HTTP request to the local Flask backend
        |-- schedule parser
        |-- OR-Tools constraint solver
        |-- Supabase repositories
                    |
              Supabase database
```

One app does not mean one process

Electron and Python are separate operating-system processes. Electron may also create renderer and graphics processes. That is normal. The current normal-close handlers stop the backend process tree owned by the app before allowing the window to finish closing. They do not stop a separately launched VS Code development server.

A concrete example

You upload a Monday class in MT102 from 7 AM to 10 AM and an assistant's personal schedule. The parser creates structured records for both. The solver checks whether the assistant is free and whether the duty satisfies the workload rules. If it selects the assignment, the interface saves a duty record linked to the original class ID. It then draws that record in the assistant's weekly calendar.

Sources: `frontend/src/desktop/main.ts`, `frontend/src/desktop/preload.ts`, `frontend/src/ui/app/App.tsx`, and `backend/app.py`.

2 How to read the code

The React hooks used throughout the interface

`useState` stores values that affect the screen. For example, selecting an assistant updates state and causes React to draw that assistant's calendar. A state update does not automatically save anything to disk; the code must call a storage helper or use an effect that saves it.

`useEffect` runs side effects after rendering, such as loading shared data, attaching a keyboard listener, or starting a polling timer. Its cleanup function removes listeners or timers when the component is no longer using them. `useMemo` avoids repeating calculations when their inputs have not changed, such as filtering a week's events. It is a calculation cache, not persistent storage.

`useRef` holds a value without forcing a redraw. Revision counters and the current CSV key are refs so asynchronous operations can compare their starting context with the latest context before applying a response.

Common JavaScript operations

`filter` keeps matching entries without changing the original array. `map` produces a new entry for each original entry. `some` answers whether at least one entry passes a condition. `find` returns the first matching entry. `reduce` combines entries into a single value, such as total assigned minutes.

The spread operator in `{ ...oldRecord, name: newName }` copies an object and replaces only its name. This matters because changing an assistant's name should not erase their duties, student ID, or personal schedule. A `Set` stores unique values, such as deleted event IDs. A `Map` connects keys to values, such as assistant ID to total minutes.

Async does not mean cloud only

An `async` function returns a Promise. `await` waits for an asynchronous result, which might be a network response, file reading, or an IndexedDB transaction. A `try` block handles the normal operation; `catch` handles failure; `finally` clears busy states even if the operation fails.

In a local-first action, the interface should wait for the local write that protects the user's work, but should not make that local write wait for an internet request. This is why archive Save now waits for IndexedDB and then closes, while cloud synchronization continues separately.

Sources: `frontend/src/ui/student-assistants/StudentAssistantPanel.tsx`, `frontend/src/storage/localFirstSync.ts`, and `frontend/src/storage/archiveWorkflow.ts`.

3 The main records and identifiers

Records that should not be confused

Record	Meaning	Important links
CalendarEvent	An imported recurring class, a dated event, or an internal exam marker	id, source, dayCode or date, room, startMinutes, endMinutes
UploadedAssistant	A student's identity and personal class schedule	id, studentId, label, events
DutyAssignment	An assistant assigned to a recurring class occurrence	assistantId, classId, day, startMinutes, endMinutes
RelieverAssignment	Date-specific coverage of someone else's duty	originalAssistantId, replacementAssistantId, classId, date
StudentAssistantResult	Current assignments, totals, relievers, settings, and solver status	assignments, optimizedAssistantIds, assistantTotals
ScheduleArchive	A complete saved snapshot and its viewing context	id, schedule, adminEvents, assistantData, view

Why the IDs matter more than the displayed names

An assistant ID links duty records to a student even when the student's name changes. A class ID links a duty to the imported class. An event ID identifies the individual card, while a booking-series key groups cards from one multi-room or multi-date booking. Recreating an ID unnecessarily can break these relationships.

The main schedule's fingerprint is a SHA-256 digest of the uploaded file's bytes. It is the normal schedule key for assistant rosters, duty-result caches, and events. Renaming the same file without changing its contents keeps the same fingerprint. Changing the bytes can produce a different fingerprint even if the file looks similar. Older saved records may use a filename-based fallback key.

Time is stored as minutes

7 AM is 420 minutes after midnight. 10 AM is 600. Their duration is `600 - 420`, or 180 minutes. Numeric times make sorting, overlap checks, workload sums, and solver constraints simpler than comparing display strings.

```
export function parseInputTime(value: string): number {
  const [hour, minute] = value.split(':').map(Number)
  return hour * 60 + minute
}
```

This helper splits an HTML time value such as `07:30`, converts the two pieces to numbers, multiplies the hours by 60, and adds the minutes. Formatting a number for the screen is a separate job.

Sources: `frontend/src/types.ts`, `frontend/src/api/studentAssistantApi.ts`, and `frontend/src/storage/archiveTypes.ts`.

4 What happens when you upload a schedule

The complete route

`ScheduleCalendar.handleUpload` gets the selected file and calls `App.uploadCsv`. That function starts two operations: parsing the file and calculating its fingerprint. It does not replace the current schedule until parsing succeeds and some usable class, TBA, room, or time information exists.

`readScheduleFile` reads CSV text with a quote-aware parser, or reads XLS and XLSX sheets with SheetJS. Spreadsheet numeric identifiers use their actual numeric values instead of currency display formatting. Rows are sent as JSON to `POST /api/schedules/parse`; the upload is not a direct file write into Supabase.

How the backend chooses a format

With `format: auto`, `parse_schedule_rows` first looks for the course-offering report header within the first 50 nonempty rows. If it finds one, it uses `_parse_course_offering_rows`. Otherwise it checks the first nonempty row for the official template's labels and calls `_parse_official_schedule_rows`. It does not blindly accept any CSV.

The course-offering parser recognizes labels such as Stub No., Course No. and Description, Time, Day, Room, Teacher, and Credits. It reads section context and metadata and combines repeated shared meetings where appropriate. Unresolved course-offering times become TBA entries rather than imaginary calendar meetings.

The official template uses labels including Subject, SubjectTitle, StartTime, EndTime, Day, RoomType, Room, StudentAmount, LastName, and FirstName. It also handles room-only and time-only rows. An incomplete class row is not necessarily treated like a room-only row: invalid required class data can reject the upload. TBA behavior depends on the parser branch.

What succeeds afterward

The interface saves the normalized classes, rooms, extra time boundaries, TBA entries, metadata, filename, and fingerprint locally. It activates that fingerprint's assistant roster and saved duty result, then queues the active schedule for cloud synchronization.

Example: a CSV row containing only Server Room can create a calendar room column, but cannot create a solver duty. A time-only row can extend the calendar's time guides without becoming a class. Neither row invents the missing subject or assistant assignment.

Sources: `frontend/src/api/scheduleParser.ts`, `frontend/src/ui/app/App.tsx`, `backend/routes/schedules.py`, and `backend/services/schedule_parser.py`.

5 The basic scheduling rules

Hard rules and preferences

A hard rule makes an assignment invalid if it is broken. A preference helps choose between valid assignments. The solver must obey personal-class conflicts, non-overlapping duties, maximum daily and weekly regular-duty minutes, and the configured gap after three continuous duty hours.

The daily and weekly settings are maximums, not required hours that every assistant must reach. An assistant can receive fewer hours when the available classes, personal schedule, or other rules leave no valid duty. Event work and reliever work are overtime in the manual workflows and are not simply added to the regular-duty maximum checks.

The overlap rule

```
function overlaps(start: number, end: number, otherStart: number, otherEnd: number) {
  return start < otherEnd && end > otherStart
}
```

The expression is true only when one interval starts before the other ends and ends after the other starts. A duty from 7 to 9 and another from 9 to 10 touch but do not overlap. A duty from 7 to 9 and an event from 8 to 10 do overlap. A separate duty-gap rule can still forbid adjacent duties even when the overlap test allows them.

The break after three hours

Suppose the required gap is 30 minutes. After a continuous 7 AM to 10 AM duty block, another regular duty starting at 10:15 AM is too early, while one starting at 10:30 AM satisfies the gap. Two adjacent duties from 7 to 8 and 8 to 10 can form the same continuous three-hour block.

The rule does not insert a break inside a class. It constrains the start of a later duty after a completed continuous block. The backend walks possible chains of adjacent duty occurrences and adds a constraint forbidding the entire conflicting combination. The manual-assignment screen checks the corresponding time intervals before offering a student.

Sources: `_add_duty_break_constraints` in `backend/solver/student_assistant_solver.py` and `violatesDutyGap` in `frontend/src/ui/schedule/ScheduleCalendar.tsx`.

6A Understanding CP SAT before reading our solver

What we are actually creating

Our app assigns student assistants to duties in an existing class schedule. It does not move the imported classes to new times, create the students' academic timetables, or choose their subjects. Their uploaded personal classes are fixed busy periods. The decisions are who covers which eligible duties.

CP means constraint programming. We describe possible decisions and rules instead of writing every possible schedule ourselves. SAT refers to Boolean satisfiability, the problem of finding true or false choices that satisfy logical conditions. OR-Tools CP-SAT combines integer constraint modeling with SAT-based reasoning and other optimization techniques. It is an optimization engine, not a trained AI model. [G1 and G2]

Four building blocks

Concept	Plain meaning	Scheduling example
Decision variable	A choice the solver may make	x equals 1 if Anna covers class C1 and 0 otherwise
Domain	Allowed values for that choice	x may only be 0 or 1
Hard constraint	A condition every accepted answer must satisfy	Anna cannot cover two overlapping duties
Objective	A numerical score used to compare allowed answers	Reward coverage and improve workload balance

We first build a `CpModel` containing these choices, restrictions, and scores. A `CpSolver` then searches that model. The model describes our problem; the solver is the general engine that finds values for its variables. Integer modeling is why our code uses whole minutes such as 420 for 7 AM. [G1]

Valid does not always mean best

A feasible schedule follows every hard constraint. An optimal schedule is feasible and has the best objective value the solver can prove for the supplied model. A schedule can be valid without being proven best. Even an optimal answer is optimal only for the rules and score we actually modeled, not for every real-world concern. [G1]

Our daily and weekly settings are upper bounds, not targets that every assistant must reach. A student can receive fewer hours, including zero. Giving a duty more preference points cannot override a personal-class conflict or another hard rule.

Sources: [G1] Google CP-SAT solver documentation; [G2] Google OR-Tools CP-SAT overview. Full links appear in section 6H. Application scope comes from runSolver and solve_student_assistant_schedule.

6B How the CP SAT engine searches

Simplify before exploring

Presolve simplifies the supplied model and can eliminate choices already determined by its rules. During search, propagation derives consequences of current choices. For example, if Anna is selected for a class that permits at most one assistant, the other assistants' variables for that class must be zero. These are logical deductions, not separately scored recommendations. [G2]

Try choices and learn from contradictions

When deductions do not finish the problem, the engine makes decisions and propagates again. If a branch contradicts a rule, it can backtrack and learn a clause that prevents the same conflicting combination from being explored again. The conceptual loop below explains this reasoning; it is not a literal trace of a run or a promise of a fixed search order. [G3]

```
Build and simplify the model
          |
Propagate consequences of current choices
          |
Unresolved choices remain ---- Make a decision ----+
          |                                        |
          +-------------- repeat propagation <---+
          |
Conflict ---- Learn a restriction and backtrack
          |
Complete valid candidate ---- Compare its objective score
          |
Continue improving or proving the best achievable score
```

Keep the best candidate and stop with a status

Optimization tracks an incumbent, meaning the best valid candidate found so far, and uses bounds to reason about what remains achievable. Search need not enumerate every complete schedule. Our app requests eight search workers; OR-Tools can use a portfolio of cooperating search strategies rather than assigning one worker to each student. The exact internal strategies depend on the OR-Tools version and model. [G2 and G4]

The run can finish with a proof or stop at its limit. Our code sets a 60-second solver search limit. That is a ceiling on the search, not a mandatory wait and not a limit on all upload, model-building, response, and saving time combined.

This is why the animated Creating schedule button cannot honestly display an exact completion percentage. The app does not stream search progress, and finding a good candidate does not reveal how much proof work remains.

Sources: [G2] OR-Tools overview and presolve components; [G3] OR-Tools SAT solver implementation; [G4] OR-Tools troubleshooting and parallel search documentation. Links appear in 6H.

6C Our process from Proceed to model inputs

Step 1 Capture the request in the interface

StudentAssistantPanel.proceedWithSchedule closes the settings dialog and calls runSolver. It checks whether generation is already running, captures the current roster, result, settings, and CSV context, and chooses full generation or incremental filling. With an existing result, unchecked full rescheduling, and no new assistants, it stops with a message instead of silently rebuilding.

solveStudentAssistantSchedule creates a random seed and sends mainSchedule, assistants, schedulingSettings, randomSeed, and optional incremental data as JSON to POST /api/student-assistant/solve. Each assistant contains an ID, label, and uploaded personal class meetings. The Flask route and service pass the validated request into solve_student_assistant_schedule.

```
Proceed
  -> runSolver captures context and generation mode
  -> solveStudentAssistantSchedule sends JSON to Flask
  -> route and service validate the request
  -> Python solver normalizes inputs and builds a CpModel
  -> CP-SAT searches the model
  -> response is checked saved locally and displayed
```

Step 2 Validate settings and normalize meetings

The Python entry point checks for a main schedule and assistants. _duty_gap_minutes accepts gaps of 0, 30, 60, 90, or 120 minutes. _workload_limits accepts daily limits from 1 to 8 hours and weekly limits from 5 to 40 hours, in hourly steps, with weekly not lower than daily. The selected settings are sent explicitly; backend defaults are only fallbacks for missing values.

_parse_meetings validates time ranges and expands weekday codes. A source class C1 with dayCode TTh becomes a Tuesday Meeting and a Thursday Meeting. Both keep source_id C1, while occurrence_id distinguishes C1-T and C1-Th. The class identifier connects those weekly occurrences to one assignment choice.

Step 3 Split meetings into coverage units

_coverage_units walks each meeting from its actual start to end in pieces of up to 30 minutes. A 7 AM to 9 AM meeting produces four units. An 8:10 AM to 9 AM meeting produces an 8:10 to 8:40 unit and an 8:40 to 9 AM unit. It does not round the meeting onto a whole-hour timetable.

For each assistant, the code also parses personal classes and any fixed duties supplied by incremental mode. Invalid personal schedules or overlapping fixed duties return INVALID before search. The units become the accounting and interval variables explained next; they are not permission to distribute a normal class arbitrarily among assistants.

Sources: StudentAssistantPanel.tsx lines 426 onward; studentAssistantApi.ts lines 99 onward; backend/routes/student_assistants.py; backend/services/student_assistant_service.py; student_assistant_solver.py lines 46 to 145 and 449 onward.

6D Our decision variables and candidate filtering

Step 4 Reject impossible assistant and class pairs

The code groups coverage units by the original source class ID. For each assistant and class, it asks whether any unit overlaps any personal class or fixed duty on the same weekday. If even one unit conflicts, `continue` skips that entire assistant and class pair. The app does not offer the student just the conflict-free half of that normal source class.

Suppose C1 occurs Tuesday and Thursday from 7 AM to 9 AM. If Anna has a personal class on Thursday from 8 AM to 9 AM, Anna is excluded from C1 altogether, even if Tuesday is free. This is the consequence of linking all occurrences with the same source ID.

Step 5 Create one class decision and its unit decisions

For every eligible pair, new_bool_var creates a class variable named uses_assistant_class. Its value starts as unknown within the allowed domain {0, 1}. Building the variable does not assign the duty yet. The value is determined when the solver searches the finished model.

Every unit of that source class also gets a Boolean assignment variable. The actual equality below links the unit's choice to the class's choice.

```
model.add(variable == class_variable)
```

If uses_Anna_C1 is 1, every linked unit is 1. If it is 0, every linked unit is 0. Thus the normal full model chooses one consistent assistant across all units and weekdays of C1. Distinct source IDs remain separate choices even if their subject names are identical.

Step 6 Make selected duties visible to overlap constraints

Each unit gets an optional fixed-size interval. Its presence is controlled by its assignment Boolean; its start and duration come from the imported timetable. The weekly position is weekday index times 1440 plus startMinutes, so Monday and Tuesday do not occupy the same point in the model.

```
Class C1 decision for Anna
          |
          +-- Tuesday unit 1 decision equals the class decision
          +-- Tuesday unit 2 decision equals the class decision
          +-- Remaining Tuesday and Thursday units follow it
          |
Each selected unit activates its fixed time interval
```

Candidate filtering handles already known personal and fixed-duty conflicts. The next constraints handle combinations of new duties that would conflict with each other. These are separate checks because two duties can each be individually available but impossible to select together.

Source: backend/solver/student_assistant_solver.py, candidate and optional-interval construction inside solve_student_assistant_schedule.

6E Our hard constraints in order

Step 7 Add the duty gap restrictions

_add_duty_break_constraints examines possible duty occurrences for an assistant on a day. It follows adjacent duties into a continuous chain. Once a chain reaches at least 180 minutes, another duty beginning before the required gap ends cannot be selected together with that chain. With a zero gap, the helper adds no such restrictions.

For a 30-minute gap, duty A from 7 AM to 8 AM and duty B from 8 AM to 10 AM form a three-hour chain. Duty C from 10 AM to 11 AM is too soon. The code adds a restriction equivalent to xA + xB + xC <= 2. At least one of those choices must be absent. A following duty beginning at 10:30 AM satisfies that particular gap.

This is a gap before a later duty, not an inserted break inside a class and not a universal three-hour maximum for a single class. The helper considers completed duty boundaries. A long individual duty can remain selectable if the other constraints allow it.

Step 8 Prevent duplicate coverage of a class

At-most-one constraints apply to both unit candidates and class candidates. For a class with Anna and Ben eligible, the class rule is equivalent to xAnnaC1 + xBenC1 <= 1. They may not both cover it. Zero is still allowed, so an uncovered class does not automatically make the whole model impossible.

Step 9 Enforce workloads and prevent simultaneous duties

For each assistant, total_minutes equals fixed weekly minutes plus the sum of selected unit durations. Its integer domain runs from zero to the weekly maximum, enforcing that cap. Per-day weighted sums separately enforce the daily maximum. For example, 120*xC1 + 180*xC2 <= 240 forbids selecting both a two-hour and a three-hour duty on a day limited to four hours.

`model.add_no_overlap(optional_intervals[assistant_id])` prevents that assistant's selected unit intervals from overlapping. Two new duties at the same time can therefore be excluded together even if neither overlaps a personal class. Fixed-duty clashes were already filtered before these optional intervals were created; fixed minutes still count toward the caps.

What these rules do not guarantee

The full model does not move room bookings or repair imported room conflicts. It constrains each assistant's time, not the physical validity of every source room record. It also does not currently receive dated event assignments as busy periods. Event conflicts checked in manual UI flows are not automatically additional constraints in this full solve.

Source: _add_duty_break_constraints and the capacity, at-most-one, and no-overlap blocks in backend/solver/student_assistant_solver.py.

6F Our objective and the search call

Step 10 Score only allowed combinations

After adding hard constraints, the code creates helper variables for workload extremes, active duty days, repeated start times, and adjacent duties. The objective combines the following terms into one score. Positive terms reward a choice; negative terms penalize it. A hard constraint still overrides every reward.

Objective term	Weight	Effect when maximizing
Selected assistant and class pairs	+1000000000 each	Strongly rewards covering more source class IDs
Lowest assistant weekly minutes	+100000 per minute	Favors improving the least loaded assistant
Highest assistant weekly minutes	-10000 per minute	Discourages concentrating work on one assistant
Total assigned minutes	+100 per minute	Rewards more covered duty time
Assistant days with duty	+4000 each	Rewards spreading duty participation across days
Assistants with an adjacent duty pair	+700 each	Gives a small preference for an adjacent pair
Selected 7 AM occurrences	-2000 each	Discourages some early starts
Repeated start times beyond the first	-1000 each	Discourages repeated identical start times
Random value for each selected class pair	+0 to 99	Helps choose among similarly scored alternatives

Coverage is counted by source class ID here, not only by hours. A two-hour class and a four-hour class each contribute one large class reward. Their minutes also affect the separate workload and time terms. This is a weighted objective, not a sequence of separate optimization passes. Its large coefficients express priority but do not constitute a formal size-independent lexicographic guarantee.

Step 11 Run CP SAT with the complete model

```
solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = 60
solver.parameters.num_search_workers = 8
solver.parameters.random_seed = random_seed
solver.parameters.randomize_search = True
status = solver.solve(model)
```

This is the point where OR-Tools searches. Everything before it described choices, rules, or scoring. The frontend supplies a fresh seed, and the model adds random tie-break points, so valid assignments can differ between full generations. The app does not train from previous schedules. Parallel search also means identical inputs should not be treated as a promise of identical output.

Sources: objective and solver configuration in student_assistant_solver.py; seed generation in studentAssistantApi.ts. General search configuration background is [G4] in 6H.

6G A small example from inputs to assignments

A simplified input that uses the same logic

Consider three assistants with a four-hour daily maximum, a twenty-hour weekly maximum, and a thirty-minute duty gap. The meetings below are duty opportunities already present in the main schedule. Personal classes are separate busy periods.

Class ID	Days	Time	Weekly duty duration
C1	Monday	7 AM to 9 AM	2 hours
C2	Monday	8 AM to 10 AM	2 hours
C3	Monday	10:30 AM to 12:30 PM	2 hours
C4	Tuesday and Thursday	7 AM to 9 AM	4 hours

Anna has a personal class Monday from 8 AM to 9 AM. Ben has one Monday from 10 AM to 11 AM. Carla has one Tuesday from 7 AM to 8 AM. Their candidate matrix becomes the following before any workload balancing.

Assistant	C1	C2	C3	C4
Anna	Excluded	Excluded	Eligible	Eligible
Ben	Eligible	Eligible	Excluded	Eligible
Carla	Eligible	Eligible	Eligible	Excluded

Anna is excluded from C1 and C2 because of Monday's overlap. Ben is excluded from all of C3, even though only its first half hour conflicts. Carla is excluded from C4 on both weekdays because Tuesday conflicts with one occurrence of that source class.

Reason about combinations rather than one student at a time

Ben cannot cover both C1 and C2 because those duties overlap. The class rules also prevent Ben and Carla from both covering C1. One valid candidate assigns C1 to Ben, C2 and C3 to Carla, and C4 to Anna. This covers all four source classes. Anna receives four weekly hours, Ben two, and Carla four. Carla's two Monday duties total four hours and have a thirty-minute gap.

Another valid candidate assigns C3 and C4 to Anna, C1 to Ben, and C2 to Carla. Its totals are 6, 2, and 2. Coverage, total minutes, and minimum workload are unchanged, but its maximum workload is two hours higher. That costs 120 times 10000 points, outweighing its extra active day's 4000-point reward. The objective favors the more balanced candidate. Similar alternatives can still swap C1 and C2 between Ben and Carla.

This is a teaching comparison, not a captured execution trace or a guarantee that a particular named student is chosen. The engine searches the joint model. It does not simply finish Anna's schedule, freeze it, and then give Ben whatever is left.

6H Reading the result and completing the process

Step 12 Read the status and selected values

OR-Tools can return OPTIMAL, FEASIBLE, INFEASIBLE, MODEL_INVALID, or UNKNOWN. For this optimization model, OPTIMAL establishes the best score; FEASIBLE supplies a valid candidate without a proof of best score. INFEASIBLE proves no allowed combination exists. MODEL_INVALID identifies a malformed model; UNKNOWN can mean a limit was reached before a conclusion. [G1]

Our wrapper accepts OPTIMAL and FEASIBLE. It currently labels all other solver outcomes INFEASIBLE, losing that distinction. Its own input checks can return INVALID before search. Also, unassigned classes are permitted: an otherwise valid full model can return a successful result with zero assignments. An uncovered class alone is not evidence of solver failure.

Step 13 Convert the chosen units into duty records

For each assignment Boolean, solver.value(variable) reads the chosen 0 or 1. Selected units are grouped by assistant and meeting occurrence. _merge_assignments joins adjoining units into a readable duty, so four chosen half-hour units become one 7 AM to 9 AM card. Tuesday and Thursday remain separate occurrence records even when one class decision selected both.

The response contains assignments, assistantTotals, optimizedAssistantIds, summary counts, diagnostics, appliedSettings, and status. runSolver checks that the CSV and saved input snapshot have not changed during search. It discards an outdated response; otherwise it updates React state, saves locally, queues cloud synchronization, and lets the calendar filter records for the chosen assistant and week.

The alternate incremental path

An incremental request first enters _solve_incremental_schedule. It validates existing assignments against current source meetings, subtracts covered intervals, and creates temporary IDs for remaining open segments. Only the newly added assistants enter a fresh CP-SAT solve. Their already assigned manual duties become fixed work. Returned temporary IDs are mapped back to original class IDs and the new records are combined with the preserved assignments.

This is a newly built smaller model, not a suspended CP-SAT search being resumed. Full regeneration may replace regular assignments; incremental generation preserves the validated old ones and fills available gaps for new assistants. The detailed mode behavior continues in Chapter 7.

Sources for the general CP SAT explanation

[G1] Google CP-SAT solver documentation for integer modeling and statuses. https://developers.google.com/optimization/cp/cp_solver

[G2] Google OR-Tools CP-SAT overview and component reference. https://github.com/google/or-tools/blob/stable/ortools/sat/README.md

[G3] Google OR-Tools SAT solver source for propagation, search, and conflict learning. https://github.com/google/or-tools/blob/stable/ortools/sat/sat_solver.h

[G4] Google OR-Tools troubleshooting documentation for worker portfolios and search limits. https://github.com/google/or-tools/blob/stable/ortools/sat/docs/troubleshooting.md

6I Lexicographic ranking and weighted optimization

Two different decision methods in our app

Our assistant recommendations use lexicographic multi-criteria sorting. Our CP-SAT schedule generation uses a weighted multi-criteria objective. These methods serve different purposes and should not be described as the same algorithm.

Lexicographic sorting compares candidates by the first criterion. Only a tie allows the next criterion to decide. A better later criterion cannot compensate for a worse earlier one. The reliever and manual-duty candidate lists use the following order after excluding unavailable assistants.

Priority	Comparison with lower values first
1	Projected weekly workload in minutes
2	Projected daily workload in minutes
3	Longest continuous duty block after assignment
4	Number of duties that day after assignment
5	Student ID in string order or assistant ID if missing

For example, Anna with 22 projected weekly hours ranks before Ben with 23, even if Ben has fewer hours that day. If both have 22 weekly hours, daily workload decides. If that is also equal, the comparison continues down the list. These are ranking examples, not a claim that a manual regular-duty assignment can exceed its workload caps; availability filtering happens first, and reliever work is overtime.

How the comparator implements this order

The sort callback subtracts the right candidate's value from the left candidate's value. A negative result places the left candidate first. The || operators return the first nonzero comparison; zero means tied, so evaluation continues down the table. Student identifiers use localeCompare instead of numeric subtraction.

This ordering recommends a candidate for one requested duty. It does not search all possible future schedules or prove that this individual choice produces the best complete schedule. Find Reliever automatically takes the first eligible candidate; the manual-duty flow lets the user choose from the ordered list.

Why our CP SAT objective is different

The solver combines coverage, workload balance, and the other terms in section 6F into one score. Weighted terms can trade off against each other. The code does not run separate optimization stages that prove the best first objective, fix it, and then optimize the next. Therefore we should not claim strict lexicographic optimization for schedule generation.

An accurate system description is: The system uses CP-SAT with a weighted multi-criteria objective for schedule generation, and lexicographic multi-criteria sorting for assistant recommendations.

Sources: frontend/src/ui/schedule/ScheduleCalendar.tsx, rankRelievers comparator at line 714 and availableRegularDutyCandidates comparator at line 779; backend/solver/student_assistant_solver.py, model.maximize at line 677.

7 Create schedule and reschedule entire schedule

The button and settings window

Create schedule opens Scheduling Settings. The user selects maximum daily hours, maximum weekly hours, and the duty gap. Proceed closes the settings dialog and calls `runSolver`. During the request, the button area becomes the animated Creating schedule display. It is an indeterminate activity indicator, not a measured solver completion percentage. Other interface actions are not globally blocked.

Two generation modes

With no existing result, the app requests a full solve. With an existing result and Reschedule the entire current schedule unchecked, it finds assistants not listed in `optimizedAssistantIds` and requests an incremental solve. If there are no new assistants, it shows a message instead of rebuilding the duties silently.

An incremental solve validates the old assignments against the current class IDs and times, keeps those assignments, subtracts their occupied intervals, and builds open-duty segments. It solves only for the new assistant IDs. If a newly added assistant already has manual duties, those duties are passed as fixed work. The response combines the old records with the newly fitted assignments.

With Reschedule the entire current schedule checked, the app does not pass the existing result as incremental input. The solver may choose different regular assignments. Reliever records are retained only when their original duty still exactly matches an assignment in the new result.

Protecting changes made while solving

Before the request, `runSolver` captures the roster, result, and settings. Afterward, it checks the current CSV key and compares the latest snapshot. If the user changed those inputs while solving, it discards the stale response instead of overwriting the user's work.

An important event limitation

Full regeneration does not delete the separately stored events. However, the present solver request sends the main recurring classes and the assistants' personal schedules; it does not send dated event assignments as busy periods. Therefore it does not yet guarantee that a newly generated duty avoids an already assigned event. Keeping event records is implemented; automatically solving around every dated event was a suggested improvement, not an implemented rule.

Sources: `StudentAssistantPanel.runSolver`, `studentAssistantApi.solveStudentAssistantSchedule`, and `_solve_incremental_schedule` in the Python solver.

8 Manual duties and assistant editing

Assigning a duty without running the solver

Clicking a class with no assigned assistant opens the manual-assignment flow. `availableRegularDutyCandidates` considers every assistant in the active CSV roster, including people with zero duty hours. It removes candidates with personal-class conflicts, effective duty or reliever conflicts, overlapping assigned events, excessive regular daily or weekly workload, or a duty-gap violation.

The list is ordered by projected weekly minutes, daily minutes, longest continuous block, duty count, and student identifier. Before saving, `assignRegularDuty` recalculates availability, verifies the class is still unassigned, and creates a DutyAssignment linked to the class ID and weekday.

`withManualDutyAssignments` rebuilds the result's assignment list, totals, and summary. It can create a FEASIBLE result even when no solver-generated result exists. Manual changes are therefore real saved schedule data, not a decoration on the calendar. The storage layer gives the changed result a new per-CSV revision.

Editing and removing assistants safely

Changing only a name preserves the assistant ID and updates labels in related duty and reliever records. `planAssistantEdit` checks replacement personal schedules against that assistant's existing commitments and returns a proposed result plus a conflict list. It does not mutate the saved records while the user is reviewing the list.

The edit form requires confirmation before saving changes that unassign conflicting work. It checks the review snapshot again so a stale confirmation does not erase newer assignments. The app shell updates or clears the assistant labels on the affected events without recreating those event cards.

Removing an assistant uses `withoutAssistantAssignments`, which filters out only that assistant's duties and reliever links and recalculates the remaining totals. It does not erase everyone else's result or delete the imported classes. Their event assignments are cleared while the event cards remain.

Sources: `frontend/src/formatters/assistantSchedule.ts`, `frontend/src/storage/studentAssistantStorage.ts`, `StudentAssistantPanel.saveEditedAssistant`, and `ScheduleCalendar.assignRegularDuty`.

9 Relievers and overtime

Finding someone to cover a duty

The reliever flow starts from a duty that already has an original assistant. `rankRelievers` excludes that person and removes students who have an overlapping personal class, event, effective duty, or existing replacement duty. The remaining candidates can exceed the regular-duty workload limits because covering a duty is treated as overtime.

Candidates are sorted by projected weekly workload, daily workload, longest continuous block, number of duties, and student identifier. Find Reliever takes the first eligible candidate, saves a date-specific RelieverAssignment, and closes the flow. It does not rerun the solver or permanently replace the original recurring DutyAssignment.

This is lexicographic multi-criteria sorting: each later criterion is used only to break a tie in the earlier ones. Section 6I explains the comparator and contrasts it with the CP-SAT weighted objective.

Original assistant and replacement assistant

For that date, the original assistant sees a relieved-duty state and the replacement sees a reliever-duty state. On a different week without that dated record, the original recurring duty still applies. This separation is why a reliever record contains both the original and replacement assistant IDs and a concrete date.

If no candidate exists, the code leaves the original assignment in place and shows No reliever available. It does not store a fake replacement or delete the original duty. Change Reliever appears after coverage exists and allows a different eligible candidate to be saved. Remove Reliever removes only that date-specific coverage record.

What ranking does and does not mean

Recommended means first according to the coded ordering, not a separately trained prediction model. The ranking's effective-workload calculations differ from the weekly report: event work is checked as a time conflict but is not added to the reliever ranking's effective weekly minute sum. The weekly report separately includes event overtime. A candidate ranking is not the same calculation as the final report total.

Sources: `ScheduleCalendar.rankRelievers`, `effectiveIntervalsFor`, `effectiveWeeklyMinutes`, `persistRelieverRecords`, `findRelievers`, and `relieverRecordFor`.

10 Events rooms dates and exam dates

One booking can create many cards

AdminEventsPanel maintains the selected dates, rooms for each date, and time ranges for each date. These are temporary form values until the user saves. Applying rooms or times copies the selected values to target dates. Select All selects the available room names; it does not invent rooms outside the current schedule.

The save operation expands the selection into a card for every date, room, and time range combination. For example, two dates, three rooms on each date, and two time ranges per date produce 12 cards. Each card has its own event ID and schedule key, so assigning an assistant to one card does not automatically assign every room and time.

Occupied rooms and conflicts

The form distinguishes scheduled classes, other events, and existing room bookings. Selecting occupied rooms displays the short warning text. Existing room bookings can block a conflicting selection; class or event conflicts can require the form's replacement confirmation. This is not the same as claiming that every occupied room can always be booked without review.

Exam dates hide classes without deleting the CSV

Exam bookings and internal exam markers encode their kind in the event ID. `isExamDate` checks whether such a record exists on a date. Calendar, weekly workload, and export code then suppress normal classes or regular duties for that date while keeping dated events visible. The source CSV class records are not physically removed.

The form can also preview exam dates while editing, so the calendar updates behind the drawer. Closing an unsaved form removes that preview; it is not a permanent class deletion. Deleting or changing the saved exam records restores the normal display where no other exam marker remains.

Title and description text are stored separately. AutoExpandingTextarea resizes the input to its content, and event details show the description when a card is opened.

Sources: `frontend/src/ui/admin-events/AdminEventsPanel.tsx`, `frontend/src/formatters/examPeriod.ts`, and the event-save handlers in `App.tsx`.

11 Assigning assistants to events and editing bookings

Availability is checked for the selected card

`availableEventAssistants` starts with the active CSV's assistants. On ordinary dates, it removes people with an overlapping personal class, regular duty, or reliever duty. On exam dates, the current rule suspends those normal class and duty checks. Overlapping event assignments are still forbidden on exam dates.

There is no normal-duty workload cap in this event-assignment filter, because event work is overtime. That does not mean an assistant can be in two rooms at the same time. Once the student is assigned to one overlapping event, they disappear from another card's available selection. An event appears in an assistant's weekly calendar only when its assistantId matches that student.

Why editing should preserve card identity

Before the fix, updating a booking deleted its old cards and created new ones, losing their IDs and assistant links. `editedBookingEvents` now matches exact unchanged date-room-time cards first. It then tries unambiguous changes to one dimension and finally a unique remaining old/new pair. Every match consumes one old card, so an assignment cannot be copied to several rooms.

Newly added cards start unassigned. When the exam kind changes, a new kind-compatible ID can be required; otherwise matched cards keep their IDs. `bookingSeriesKey` strips numeric/date suffixes to keep cards in a recognizable booking series.

Revalidation and confirmation

`validateEditedEventAssistants` rechecks changed slots against personal classes, regular duties, reliever duties, and the final set of scoped events. Text-only edits preserve assignments without needlessly changing availability. Removed assigned cards and invalid retained assignments are listed in a confirmation prompt.

If the user cancels, nothing is deleted and the editor stays open. If confirmed, App commits one new local calendar snapshot and queues the changes together. Cloud event requests are still individual requests, not a multi-card database transaction.

Sources: `frontend/src/formatters/bookingEdit.ts`, `frontend/src/formatters/bookingSeries.ts`, `App.commitEventEdit`, and `ScheduleCalendar.availableEventAssistants`.

12 How the calendars are drawn

Dates are views over stored data

The Schedule tab's daily view places rooms across the top and times down the side. Weekly view uses dates for the selected room. ScheduleDateNavigation combines previous/next navigation with the clickable date heading and date-picker controls. Moving the date does not generate new assignments.

The assistant calendar expands recurring weekday codes and combines that person's personal classes, regular duties, assigned dated events, and relevant reliever records. The selected Monday determines the seven displayed dates. An event outside that week is filtered out even if it belongs to the same assistant.

Turning minutes into positions

Both calendars convert elapsed minutes from the visible start into vertical positions. The scale uses a row height and the current zoom. A block's height follows its duration, so a three-hour duty is taller than a one-hour event. The assistant view also considers the available viewport height when choosing its base scale.

CalendarZoomControls changes the value from 75 percent to 200 percent in 25-point steps; the middle button resets it to 100 percent. Zoom changes presentation, not class times or solver inputs. The two tabs maintain their own view state.

Conflict colors are not solver instructions

The Schedule tab compares pairs of visible events in the same room. It calculates `max(start times)` and `min(end times)`. If the first value is smaller than the second, it stores the overlapping interval and shows the warning or conflict color. Weekly conflict checks repeat this per day.

Room-time conflicts in the imported data are different from an assistant's personal availability conflict. A red room card indicates conflicting room records; it is not proof that the solver assigned one student twice.

View from an assistant duty sends a class ID and date to App. The Schedule tab focuses and briefly highlights that card. The focus request is marked consumed and cleared according to the interaction logic; it is not a permanent assignment flag.

Sources: `ScheduleCalendar.tsx`, `AssistantWeeklyCalendar.tsx`, `ScheduleDateNavigation.tsx`, and `CalendarZoomControls.tsx`.

13 Weekly workload reports

The report includes the whole roster

WeeklyWorkloadSummary shows a ranked list rather than a separate report of all individual duty descriptions. `weeklyWorkloads` creates a row for every assistant, including people with zero assigned minutes. Each row has regular-duty, event, reliever, and total minutes.

For recurring duties, the function maps the assignment's weekday onto the selected week. Exam dates do not contribute regular-duty minutes. Dated events count only when the date is within that week and the assistantId matches a roster member. Internal exam markers are not event overtime.

Avoiding double counting when someone is relieved

If another student covers part of a duty, the original assistant's credited duty minutes are reduced by the actual covered interval. The function intersects each reliever interval with the duty and merges overlapping coverage before subtracting it. This prevents the same covered time from being subtracted twice.

The replacement receives reliever minutes for their date-specific record. For example, if Maria has a three-hour duty and Juan covers its middle hour, Maria's effective duty total is two hours and Juan receives one reliever hour in the report. This reporting capability does not imply that every manual UI path allows creating arbitrary partial coverage records.

Sorting the results

The total is regular-duty minutes plus event minutes plus reliever minutes. `sortWeeklyWorkloads` returns a sorted copy, descending by default or ascending when selected. Equal totals are ordered by name and then ID, which makes the display stable rather than randomly shuffling tied students.

The report does not write assignments back to the solver and does not change the schedule. It is a calculated view over the selected week's data.

Sources: `frontend/src/formatters/weeklyWorkload.ts` and `frontend/src/ui/student-assistants/WeeklyWorkloadSummary.tsx`.

14 Archives are saved snapshots

Capturing and saving

createScheduleArchive validates that a schedule is open and that archive details are present. It captures the imported schedule, current CSV's event records, relevant assistants and duty result, settings, and viewing dates. It scopes roster and result caches to that schedule rather than copying unrelated CSV histories.

The snapshot is cloned through JSON serialization so later changes to the active objects cannot silently alter the archive. Its UUID and schemaVersion identify the snapshot and its supported structure. Large archive snapshots are stored in the separate auto-scheduler-archives IndexedDB database.

```
// Closing the working schedule must never happen before the snapshot commits.
export async function saveArchiveWithAction(archive: ScheduleArchive, afterSave?: () => void) {
  await saveArchiveLocally(archive)
  afterSave?.()
}
```

The await protects the local commit. Only after IndexedDB succeeds does the supplied close action run. App then clears the current working schedule and closes the archive manager. A failed local save does not call that action. Cloud synchronization is started separately and cannot delay this local save merely because a request is slow.

Viewing is separate from editing current work

ArchivePreview uses the archived snapshot as an explicit data source and sets readOnly on the calendar. ArchivedAssistantPanel uses the same captured assistant data. Creating schedules, assigning students, deleting events, and writing normal view preferences are disabled in those preview paths. Viewing an archive does not activate its CSV in the normal working storage.

Names and offline conflicts

Local archive names are checked inside the save transaction after trimming, collapsing spaces, and ignoring case. Supabase has the corresponding unique active-name index. A name can still be claimed on another device while this device is offline.

If that happens, the local archive remains viewable with nameConflict set. Other archives continue syncing. Rename changes only that rejected, never-uploaded snapshot's name and retries. Synced archive snapshots remain immutable. Deleting a conflicting local archive does not delete the other device's differently identified archive.

Sources: `archiveTypes.ts`, `archiveStorage.ts`, `archiveWorkflow.ts`, `ArchiveManager.tsx`, `ArchivePreview.tsx`, and `ArchivedAssistantPanel.tsx`.

15 Exporting a room schedule to Excel

What the workbook contains

createRoomScheduleExcel creates one worksheet per distinct room. It exports the selected week's Monday through Friday dates, not all seven calendar days. Empty rooms can still have worksheets. Normal recurring classes are omitted on exam dates, while valid dated events remain eligible for export.

The function sanitizes worksheet names because Excel forbids some characters, limits names to 31 characters, and requires case-insensitive uniqueness. It adds suffixes when needed. Time boundaries include regular half-hour guides and every actual event start and end so non-round times can be represented accurately.

Why the export builds XML

The implementation writes the XLSX package's worksheet, style, relationship, workbook, and content-type XML parts. SheetJS's CFB utilities package those parts into a ZIP-based workbook. This gives the code control over merged schedule blocks, row heights, black borders, wrapped text, and color fills.

Events in the same room and day are sorted and grouped when their intervals overlap. Conflict groups are written with a red fill and include their individual times. Non-conflicting occupied cells use the ordinary schedule style. Touching boundaries alone do not produce an overlap group.

Downloading without changing data

downloadRoomScheduleExcel builds a browser Blob from the workbook bytes, creates a temporary object URL, clicks a download link, then revokes that URL. It does not modify the source CSV, the assignment result, or the Supabase tables.

The existing export text lists course, identifier, instructor, and room. It does not automatically mean the workbook includes every assistant or reliever detail visible in the app; those fields would need explicit export formatting code.

Source: `frontend/src/export/roomScheduleExcel.ts`.

16 Local storage cloud queues and CSV isolation

Where data is kept

Location	Main content	What to remember
React state	Current screen values and calculated views	Lost when the process exits unless persisted
localStorage	Active schedule, assistant caches, events, preferences, retry queues	Device and application-origin specific
IndexedDB	Complete archive records and pending archive sync state	Separate archive database on this device
Supabase	Shared schedule, scoped rosters and results, events, archives, deletion records	Remote persistence after successful synchronization

How retry queues work

A queue helper serializes the change locally and starts a background flush. A failed request leaves the pending entry available for later retry. The flushing functions share an in-flight Promise so several callers do not start overlapping copies of the same queue operation.

After a successful request, the code rereads the latest queue before removing anything. It clears the entry only if it still matches what was sent. If the user edited again while the request was running, the newer value stays pending.

```
const pending = readJson<SharedSchedule<CalendarEvent> | null>(PENDING_SCHEDULE_KEY, null)
if (!pending) return
await saveSharedSchedule(pending)
const latest = readJson<SharedSchedule<CalendarEvent> | null>(PENDING_SCHEDULE_KEY, null)
if (sameValue(latest, pending)) localStorage.removeItem(PENDING_SCHEDULE_KEY)
```

The pending variable is the request's snapshot, not a live object. The comparison with latest is the protection against acknowledging away a newer edit. `finally` releases the in-flight lock whether the network call succeeds or fails.

Isolation does not mean one database table per CSV

Events carry scheduleKey. Student assistant data has rostersBySchedule and resultsBySchedule maps keyed by the fingerprint. Uploading B activates B's roster and result; it does not erase A's cache. Legacy unscoped assistants are preserved in an unlinked pool, but the old linking button has been removed. They are not automatically added to every new CSV roster.

The cloud shared_schedules record still represents one shared active schedule, using the fixed ID ccs-main. Per-CSV assistant and event histories are scoped, but the system is not a fully independent multi-user workspace system. Editing the shared active schedule from another device can affect which schedule is loaded.

Sources: `frontend/src/storage/`, `App.tsx`, and the schedule and student assistant Supabase repositories.

17 Versioned assignments and permanent event deletion

Why roster versions alone were insufficient

Two devices can have the same assistant list but different duty assignments. Protecting only rosterRevisions does not tell the backend which assignment result is newer. The application now stores independent resultRevisions for each CSV.

A real result change receives a new timestamp-and-UUID revision. Switching CSVs or hydrating remote data does not mint a new edit. Frontend and backend merge logic compare these per-CSV revisions so an old cached A result included in a B save does not overwrite A's newer duties. A clear retains its revision even when the result itself is absent.

Offline edits to several CSVs remain in the cached maps and can be sent in one pending snapshot. The backend accepts newer revisioned results rather than treating every cached result as a new write. Roster revision checks still prevent a rejected older roster from restoring its assignments.

Why a deleted event needs a record

Hard deletion alone leaves no evidence distinguishing never existed from existed and was deleted. Migration 010 creates admin_event_deletions. The delete RPC records the ID and removes its event in one transaction, even if the event has not yet been uploaded.

A database trigger blocks a later insert or update using a deleted ID. It also records IDs for direct deletes from admin_events. A per-ID transaction lock serializes cooperating deletion and write checks. The backend converts a deleted-event save into HTTP 410; the frontend removes that stale queued PUT instead of retrying it forever.

Sync snapshots include deleted IDs. Local storage keeps those IDs, filters stale cached cards, and notifies the calendar to remove known-deleted cards. Ordinary network errors are different: they retain the pending change for retry. Deletions made before migration 010 cannot be reconstructed automatically.

Limits of the protection

Per-CSV revisions protect sequential synchronization against stale caches. The assistant repository still reads and writes a shared JSON envelope; it is not a transactional collaborative editor with conflict dialogs for simultaneous multi-device edits. Device clock differences are also a limitation of timestamp-based ordering. Do not promise safe simultaneous editing based only on these fixes.

Sources: `studentAssistantStorage.ts`, `localFirstSync.ts`, `student_assistant_repository.py`, `admin_event_repository.py`, and migration 010.

18 The backend routes and database tables

The layers on the Python side

A route receives an HTTP request and chooses a response status. A model validator checks the payload's shape and required data. A service connects that request to the relevant operation. A repository maps application fields to database fields and performs Supabase REST calls. The solver is a separate computational component, not a database repository.

For example, updating an event follows routes/admin_events.py, services/admin_event_service.py, models/admin_event.py, and database/supabase/admin_event_repository.py. The validator checks required event fields and times; the repository checks deleted IDs and maps courseCode to title and scheduleKey to schedule_key.

Main endpoints

Request	Purpose
GET /api/health	Reports local service health and whether cloud credentials are configured
POST /api/schedules/parse	Normalizes supplied spreadsheet rows
GET or PUT /api/schedules/shared	Loads or saves the shared active schedule
GET /api/admin-events	Returns ordinary event records
GET /api/admin-events?include_deleted=true	Returns events plus deletion metadata
PUT or DELETE /api/admin-events/id	Saves an individual event or permanently records its deletion
GET or PUT /api/student-assistants/shared	Transfers scoped rosters, results, settings, and revisions
POST /api/student-assistant/solve	Runs full or incremental constraint solving
GET /api/archives	Returns shared archive records and deletion states
PUT or DELETE /api/archives/id	Creates an immutable snapshot or marks its deletion

Cloud storage layout

shared_schedules stores the active imported schedule in a JSON envelope. student_assistant_schedules stores a scoped cache envelope in solver_result, alongside the compatibility assistants field and scheduling settings. admin_events stores individual event cards with dates, rooms, times, assistants, descriptions, and schedule keys. schedule_archives stores complete snapshots separately. admin_event_deletions stores permanently deleted event IDs.

The Supabase client uses HTTP with a 15-second timeout and raises RuntimeError on service errors. Flask maps RuntimeError to 503. Validation errors commonly return 400; archive name conflicts return 409; rejected saves for deleted events return 410.

18A Understanding the database design

Why some information is inside JSON

The database uses a mixed design. Events have individual rows with ordinary columns for their title, date, room, and time. Imported timetables and assistant results contain nested lists and maps, so they are stored in JSONB columns. Seeing only a few rows in Supabase does not mean the app has only a few students or assignments. One JSON value can contain a complete roster and hundreds of duties.

The schema below describes the project's migrations 001 through 010. The SQL files define the intended structure. A running Supabase project has that structure only after its migrations have succeeded. A source file is not proof that a remote migration ran.

Reading database types and rules

Database term	Plain language meaning
text	A string such as a title, room name, or identifier
integer	A whole number, such as minutes after midnight
date	A calendar date without a clock time
timestamptz	A timestamp representing an instant with timezone support
uuid	A structured unique identifier used for archives
jsonb	A structured object or list stored in PostgreSQL
primary key	The value that uniquely identifies a row
not null	The column cannot contain SQL NULL
default	A value used when an insert omits the column
index	A lookup structure; a unique index can also reject duplicates
trigger	Database code automatically run for a specified write
policy	A rule controlling which roles can access rows

An empty JSON array `[]`, an empty JSON object `{}`, an empty string `''`, and SQL NULL are different values. For example, an empty assistant list means the roster has no entries. A missing result can mean that no assignment result is currently saved. Neither means an assistant called NULL exists.

The relationships are mostly maintained by the app

The schedule fingerprint identifies an uploaded file's scope. Assistant IDs identify people within the stored records. Class IDs identify imported meetings. Duty records link assistantId to classId. Event rows carry schedule_key and, when assigned, assistant_id. Reliever records link the original and replacement assistants to a dated duty.

These are logical relationships, not a set of SQL foreign keys connecting normalized student, class, and assignment tables. Most related records are nested in JSON. The interface, formatters, validators, and repositories must keep their IDs consistent. A label is display text and must not be used as a substitute for a stable ID.

The two main shared records use the fixed ID `ccs-main`. That ID is not the uploaded file fingerprint. Think of ccs-main as the shared storage slot and the fingerprint as the key used inside or alongside its content to distinguish schedules.

The five tables at a glance

Table	Stored content	Record granularity
shared_schedules	Active imported timetable	Shared active slot
student_assistant_schedules	Rosters, duties, relievers, settings, revisions	Shared JSON cache slot
admin_events	Dated event cards and exam records	Individual saved event card
schedule_archives	Independent historical snapshots	One archive
admin_event_deletions	IDs of permanently deleted events	One deleted event ID

There is no separate weekly-report table. Reports are calculated from saved duties, events, replacements, and the selected week. There is also no physical table created each time another CSV is uploaded.

Sources: `database/supabase/migrations/001_shared_schedule.sql` through `010_admin_event_deletions.sql`; `frontend/src/types.ts`; the Supabase repositories under `backend/database/supabase/`.

18B The active timetable and assistant tables

The shared schedules columns

Column	Type and rule	Meaning
id	text primary key	The repository currently uses ccs-main
csv_name	text not null	Display filename; default empty string
csv_events	jsonb not null	Current timetable envelope; SQL default is an empty array for compatibility
admin_events	jsonb not null	Legacy event storage; default empty array
updated_at	timestamptz not null	Timestamp with default now

The current schedule repository writes an object into csv_events. It can still read an older array-based value. The legacy admin_events column is not the current destination for ordinary event saves; those use the separate admin_events table.

The timetable envelope contains these fields:

JSON field	Meaning
events	Normalized imported CalendarEvent records
rooms	Imported room names, including accepted room-only input
times	Imported numeric time markers
fingerprint	SHA-256 identifier for the uploaded file bytes
tbaSubjects	Unresolved subject information when supplied
metadata	School year and semester when supplied

The repository maps csvName in an API payload to csv_name in SQL, and maps csvEvents into the envelope's events field. These naming differences do not create extra copies of the original spreadsheet file.

The student assistant schedules columns

Column	Type and rule	Meaning
id	text primary key	The shared assistant record uses ccs-main
assistants	jsonb not null	Active roster compatibility field; default empty array
solver_result	jsonb nullable	Result cache envelope or an older direct result
scheduling_settings	jsonb not null	Workload and gap settings added by migration 005

Column	Type and rule	Meaning
updated_at	timestamptz not null	Timestamp with default now

The column called solver_result is now a container for per-schedule data. It must not be described as only the last raw response from CP SAT.

Envelope field	Meaning
kind	Identifies the schedule-result-cache representation
activeScheduleKey	Fingerprint of the currently active roster and result
activeResult	Convenience copy of that schedule's saved result, or null
resultsBySchedule	Map from schedule keys to saved assignment results
rostersBySchedule	Map from schedule keys to assistant lists
rosterRevisions	Version markers for real roster edits
resultRevisions	Independent version markers for assignment changes and clears
unlinkedAssistants	Preserved legacy profiles whose schedule scope is unknown

A simplified example shows the relationship without listing full profiles:

```
rostersBySchedule
  file key A -> [Anna profile, Ben profile]
  file key B -> [Carla profile]

resultsBySchedule
  file key A -> Anna and Ben duty result
  file key B -> Carla duty result
```

When file B is active, Anna and Ben should not appear merely because they are present somewhere in the shared JSON. The app activates B's roster. The former legacy-linking button is no longer available; preserved unlinked data is not an automatic roster for a new file.

Scheduling settings and actual defaults

The current defaults are a 30-minute gap, a 4-hour daily regular-duty maximum, and a 20-hour weekly regular-duty maximum. An earlier mockup showing 8 daily hours is an example of a selectable value, not the current default.

```
{
  "minimumGapAfterThreeHourDutyMinutes": 30,
  "maximumDailyDutyMinutes": 240,
  "maximumWeeklyDutyMinutes": 1200
}
```

The available gap choices are 0, 30, 60, 90, and 120 minutes. Daily choices run from 1 through 8 hours. Weekly choices run from 5 through 40 hours. Normalization rejects unsupported values and ensures the weekly setting is not below the daily setting. These settings are stored alongside the shared assistant cache, not as

independently keyed historical settings for every uploaded file. A saved result can also record its applied settings, and an archive captures its saved settings.

Timestamp defaults are not automatic update triggers

A column declared default now receives that default when an insert omits it. That declaration alone does not automatically change updated_at on every later update. Do not use the timestamp columns as proof of universal last-write ordering. The assistant cache uses explicit per-schedule revisions for its stale-cache checks.

Sources: migrations 001, 003, and 005; `backend/database/supabase/schedule_repository.py`; `backend/database/supabase/student_assistant_repository.py`; `frontend/src/api/studentAssistantApi.ts`.

18C The records inside the JSON

Imported classes and personal classes

CalendarEvent represents department classes, personal classes, and dated events. Optional fields accommodate their differences.

Fields	Meaning
id and source	Stable record identity and whether its source is csv or admin
courseCode and subject	Course or event title information
startMinutes and endMinutes	Numeric time boundaries
dayCode and date	Recurring weekday code or a concrete date
room	Location used for the calendar
classType and section	Class information when present
sections	Multiple associated sections when represented
stubCode	Imported class identifier when present
credits and studentCount	Imported academic information when available
instructorLastName and name fields	Available instructor display information
scheduleKey	Uploaded schedule ownership for scoped records
description	Event detail text
assistantId and assistantLabel	Assigned event assistant identity and display label

A personal-class record describes when a student is busy. It does not, by itself, give that student duty hours. Likewise, credits are academic units and do not mean the same thing as the number of timetable hours. The solver's duty workload is calculated from time intervals.

An uploaded assistant profile

UploadedAssistant has id, label, fileName, and events. It can also have studentId, lastName, firstName, and middleName. The events list is the student's personal class schedule. The generated duty list is stored separately in a result.

The internal id links the profile to duties and events. studentId is the school's identifier; label is the displayed name. Renaming preserves identity and updates related labels. Creating another profile creates a different record.

A regular duty assignment

Field	Meaning
assistantId and assistantLabel	The assigned student and display text
classId	The original imported class record
day	Recurring weekday
startMinutes and endMinutes	Assigned interval, which can be a segment in incremental results
courseCode and subject	Descriptive class information
room and section	Location and section information

classId lets View find the original class. Removing a duty leaves that class available for reassignment.

A saved result

StudentAssistantResult contains status and diagnostics, with optional assignments, assistantTotals, optimizedAssistantIds, relieverAssignments, appliedSettings, and summary. The summary can contain assistantCount, coverageHours, capacityHours, assignmentCount, assignedClassCount, and unassignedClassCount.

assignments is the working regular-duty list. assistantTotals summarizes regular work. optimizedAssistantIds records which assistants have already been processed for incremental generation; it is not a list of students guaranteed to have nonzero work. appliedSettings records the settings associated with the saved result when supplied.

Manual changes can save a FEASIBLE result without calling CP SAT. This does not imply a new solver proof. Use current assignments and calculated reports; retained diagnostics and historical solver metrics are not necessarily recomputed after manual edits.

A date specific reliever record

RelieverAssignment stores date, classId, day, startMinutes, endMinutes, courseCode, room, originalAssistantId, originalAssistantLabel, and optional replacementAssistantId and replacementAssistantLabel.

The original recurring duty remains in assignments. The dated record says who covers it on that date. This is why changing a reliever does not permanently change next week's original assignment. A reliever record is stored inside the assistant result, not in a separate SQL reliever table.

Local names and remote names

LocalAssistantData calls the active result result, its settings settings, and its result map resultsBySchedule. The API uses solverResult, schedulingSettings, and solverResultsBySchedule. The repository converts these into the SQL envelope described in Section 18B. These are translations of the same concepts across layers, not three independent scheduling engines.

Sources: `frontend/src/types.ts`; `frontend/src/api/studentAssistantApi.ts`; `frontend/src/storage/studentAssistantStorage.ts`; `frontend/src/formatters/assistantSchedule.ts`.

18D Events archives and deletion records

The admin events columns

Column	Type and rule	Meaning
id	text primary key	Stable event card ID
title	text not null	Displayed event title
event_date	date not null	Date of this card
room	text not null	Room for this card
start_minutes	integer not null	Start minutes after midnight
end_minutes	integer not null	End minutes after midnight
updated_at	timestamptz not null	Timestamp with default now
assistant_id	text nullable	Assigned student's internal ID
assistant_label	text nullable	Assigned student's display name
description	text not null	Description; default empty string
schedule_key	text nullable	Fingerprint ownership added by migration 008

The event repository maps the app's courseCode to SQL title, date to event_date, scheduleKey to schedule_key, and camelCase time and assistant fields to their snake_case counterparts. There is an index named admin_events_schedule_key_idx on schedule_key.

One event form can produce several rows. Two dates with three rooms and one time range create six cards. Each can have its own assistant. A null assistant_id means no assistant is assigned to that card. A null schedule_key is legacy unscoped information, not a command to show that card in every uploaded schedule.

Exam markers use internal ID conventions in the same event system. Some records support calendar behavior rather than representing ordinary visible event work. The app excludes internal markers from event overtime totals.

The schedule archives columns

Column	Type and rule	Meaning
id	uuid primary key	Snapshot identity
name	text not null	Human-readable archive name
school_year	text not null	Saved school year
semester	text not null	Saved semester
created_at	timestamptz not null	Creation instant supplied with the archive

Column	Type and rule	Meaning
snapshot	jsonb not null	Complete saved ScheduleArchive object
deleted_at	timestamptz nullable	Null while active; timestamp when deleted

The snapshot has schemaVersion, id, name, schoolYear, semester, createdAt, schedule, adminEvents, assistantData, and view. schedule contains the imported data. assistantData contains the relevant scoped roster, assignments, replacements, settings, and revisions. view contains scheduleDate, assistantWeekStart, selectedAssistantId, and tab.

The local IndexedDB ArchiveRecord wraps that snapshot with id, deletedAt, pending, and optional nameConflict. These local synchronization fields are not all SQL columns and are not all part of the immutable snapshot itself.

The archive trigger prevents changing an existing snapshot's identity, name, school year, semester, creation time, or captured data. It also prevents clearing a deletion timestamp to restore a deleted archive. The unique active-name index ignores case and repeated spaces. App deletion normally sets deleted_at rather than removing the row physically.

The admin event deletions columns

Column	Type and rule	Meaning
event_id	text primary key	Permanently deleted event ID
deleted_at	timestamptz not null	Deletion timestamp; default now

The protected delete RPC records the ID and deletes the active event in one transaction. The event-table trigger records direct row deletions too and blocks a subsequent insert or update for a deleted ID. Anonymous and authenticated clients can read deletion records but cannot directly maintain that table through ordinary writes.

A stale save for a deleted event produces HTTP 410 through the backend. The frontend drops that obsolete pending save. A new event needs a new ID; erasing deletion records to revive old IDs would defeat the protection.

Sources: migrations 002, 004, 006, 007, 008, 009, and 010; `frontend/src/storage/archiveTypes.ts`; `backend/database/supabase/admin_event_repository.py`; `backend/database/supabase/archive_repository.py`.

18E Following one change through every layer

Adding a student assistant

The user submits a profile and a personal schedule. The parser returns normalized personal-class records. The Student Assistant panel creates the profile and updates the active fingerprint's roster. Local storage saves the roster and a new roster revision. The retry queue sends a shared-assistant payload to Flask. The repository merges eligible per-schedule changes into the shared JSON envelope and saves it to Supabase.

When another device loads that envelope, its active schedule key determines which roster to show. A profile being present somewhere in the table is insufficient: it must be inside the correct roster map entry.

Creating or manually changing duties

Create schedule sends a solver request to the local backend. CP SAT returns assignment objects. A successful, still-current response becomes a local saved result, receives a result revision, and enters the cloud queue. A manual assignment takes a shorter route: it validates eligibility, rebuilds the working result locally, and uses the same result persistence path without running the solver.

The assignments are not embedded into the original CSV file. Reopening that file on another computer without the app's saved data does not reveal the generated duties by itself.

Saving a dated event

The booking form expands date, room, and time selections into event cards. The app stores those cards locally with scheduleKey. Each queued event save reaches the event route, validator, service, and repository. The repository writes one SQL row per card. The assistant calendar displays a card only for the matching assistant ID and selected date range.

Replacing a duty assistant for one date

Find Reliever filters and ranks eligible candidates, takes the first, and saves a dated replacement record inside the assistant result. The original assignment stays in place. The original student's calendar displays relieved duty for that date; the replacement's calendar displays reliever duty. Removing coverage removes the dated record rather than deleting the original class or recurring duty.

Archiving and closing current work

The app clones a snapshot and commits it to IndexedDB. Only a successful local commit allows the workflow to close the active schedule. Archive cloud synchronization is independent and can complete afterward. The historical snapshot does not depend on the active ccs-main schedule remaining open.

Loading and retrying

Startup can load saved local data before a remote request finishes. Polling callbacks normally run at five-second intervals, but that is not a promise that all devices converge within five seconds. Pending writes, backend availability, request time, failures, and merge rules affect when another device receives a change.

The queue acknowledges the version it actually sent. If another local edit occurs during that request, the newer pending value remains. Per-schedule roster and result revisions prevent several stale-cache overwrite cases. They do not make a read-modify-write JSON save transactionally safe for unrestricted simultaneous editing.

Sources: `frontend/src/storage/localFirstSync.ts`; `frontend/src/api/apiClient.ts`; `frontend/src/storage/archiveWorkflow.ts`; `frontend/src/ui/app/App.tsx`; the assistant and event repositories.

19 Desktop startup shutdown and building

Development and packaged mode

In development, npm run dev starts Vite, Electron, and Flask as separate development commands. Electron loads http://localhost:5173. In the packaged app, Electron starts the bundled auto-scheduler-backend executable and loads dist-react/index.html. The preload bridge exposes the local Flask URL to the isolated interface.

The packaged startup currently opens the window without waiting for the health endpoint. The backend still uses fixed port 5000; dynamic ports and single-instance management were discussed but are not implemented in this source snapshot. An old backend occupying that port can make a new interface talk to the wrong parser version.

What normal closing now does

mainWindow's close handler and app's before-quit handler call stopFlask. On Windows, stopBackendProcess uses a synchronous taskkill command for the exact spawned backend PID with the full-tree option. It waits for the command to finish, including the child process created by the one-file Python executable. It does not kill all Python processes by name.

If cleanup fails, the close is prevented and an error is shown rather than deliberately leaving an unowned backend behind. Development's separately started Flask server is not owned by this Electron child reference. Forced termination or a crash can bypass normal close handlers; those cases are not equivalent to clicking X.

Packaging and startup speed

npm run dist:win builds the Python executable with PyInstaller, transpiles Electron code, builds React, and runs electron-builder for a portable executable and MSI. The current backend uses onefile packaging, and the portable app also extracts resources. This helps explain startup delay. Folder-based backend packaging and deferred solver loading have not been implemented.

The package currently copies .env as a resource. The frozen backend looks for it next to the packaged backend resource folder's parent. The latest source is not automatically inserted into an old executable: rebuild after code changes and wait for packaging to finish.

Sources: `frontend/src/desktop/`, `scripts/build-backend.mjs`, `frontend/electron-builder.json`, `frontend/vite.config.ts`, and `package.json`.

20 Testing and diagnosing failures

What the automated checks prove

npm test runs Python backend tests, frontend/helper regression tests, and desktop shutdown unit checks. The tests use synthetic fixtures and mocked database or network calls rather than changing live Supabase records. npm run build validates TypeScript and creates the production interface; npm run transpile:electron validates and emits the desktop entry points.

The optional Windows shutdown integration test creates its own parent and child processes and checks that both stop. The Electron smoke harness uses an isolated profile and synthetic backend responses to exercise real interface actions, including assistant edits, event edits and deletion, and archive saving while cloud requests are deliberately held.

Passing these checks is not the same as testing the distributed executable against a real database on two devices. Packaged startup, cloud migrations, actual network credentials, and real two-device use still need appropriate verification. Existing unrelated lint findings should not be described as newly fixed merely because targeted files pass lint.

A practical debugging order

Symptom	First things to check
Unknown schedule format	Confirm the actual file headers and the running backend's parser version; check for an old process on port 5000
CSV parses but no assistant roster appears	Confirm the active fingerprint and scoped roster; legacy unlinked records do not automatically belong to this CSV
Event is local but absent from Supabase	Check pending sync, backend errors, credentials, and required migrations including 010
Old event reappears	Verify migration 010 and deleted-ID propagation; distinguish historical unrecorded deletions
No valid manual duty candidate	Check personal classes, duties, events, regular limits, and the duty gap
No reliever candidate	Check overlapping commitments; do not assume a regular workload maximum blocked overtime
Archive name cannot sync	Look for Rename on the locally saved conflict record; preserve the snapshot
New code does not appear in the app	Check that the rebuilt executable is the one being opened and that an old backend is not answering

Safe testing habit

Use test CSVs with visibly different contents and different students when checking isolation. Do not delete real Supabase rows merely to make a test pass. Back up meaningful schedules as archives, and remember that clearing app data also removes unsynced local records.

21 Questions you should be able to answer

Why use a solver instead of randomly assigning assistants

The solver searches combinations while enforcing availability and workload constraints. Randomness is used only to help choose among similar model choices; it does not replace the constraints.

Does maximum workload mean every student must receive that many hours

No. It is an upper bound. Personal schedules and available class duties can leave a student with fewer assigned hours, including zero.

Can the app work without internet

Local viewing and local saves can work. Parsing and schedule generation still need the bundled local Flask service. Remote sharing needs internet and a working Supabase project. A paused cloud project does not mean the locally stored schedule has disappeared.

Does removing the CSV delete its original file

No. removeCsv clears the application's active view and active schedule selection. The app does not write generated duties into the source CSV on disk. Duties, rosters, and events are maintained in application storage under the file's fingerprint.

Does an archive restore an editable current schedule

No. Current archive viewing is read-only. It gives historical preview and supported exports without replacing active work. There is no general Restore to Current feature in the present code.

Are the scheduling settings applied to overtime

The daily maximum, weekly maximum, and duty gap constrain regular duty generation and manual regular duty assignment. Reliever and event availability still prevent overlapping commitments, but those overtime flows do not simply enforce the same regular-duty caps.

Can we guarantee all events stay compatible after a full reschedule

Not yet. Events stay saved, but dated event commitments are not included as busy periods in the full solver request. That is a remaining design limitation, not a completed feature.

What should be explained honestly about syncing

The app queues changes locally and retries shared saves. Scoped revisions and event deletion records address specific stale-data failures. It is still a shared prototype with limited conflict handling, not a fully transactional multi-user collaboration system.

Is the database ready for public deployment

The current policies allow shared prototype access to anonymous or authenticated clients. These are not per-user authorization boundaries suitable for a public deployment. Bundling an environment file does not make its contents secret; do not distribute privileged service-role credentials.

22 A worked example from upload to archive

Prepare the example

Assume file A contains CCS 1001 in MT102 on Monday from 7 AM to 10 AM, and CCS 2200 in MTCL1 on Tuesday from 1 PM to 4 PM. Anna has a personal class Monday from 8 AM to 9 AM. Ben is free for the Monday duty. Carla will be added later. These are teaching examples, not a promise that a larger real dataset produces exactly these assignments.

Upload file A

The app reads the file into rows and asks the backend to recognize and parse its format. It calculates a fingerprint from the actual uploaded bytes. Successful parsing populates the Schedule tab and selects that fingerprint's assistant roster and saved result, if they exist.

The original file stays unchanged on disk. A previously unknown fingerprint can start with an empty assistant roster even when other students exist in the database under a different key. The filename is useful for people, but the fingerprint is the normal isolation key.

Add Anna and Ben

Each profile stores personal classes separately from department duties. Anna's Monday conflict makes her unavailable for the overlapping Monday assignment. Ben may be eligible. Neither person receives work merely because their profile appears in the sidebar.

Choose Create schedule and review settings. With no existing result, Proceed requests a full solve. If the solver assigns Ben to Monday, the saved duty references Ben's internal assistant ID and the original CCS 1001 class ID. It also stores the weekday and interval needed to draw the duty.

Review and make a manual change

Suppose Tuesday is unassigned and Anna is free then. Clicking the Tuesday class lets the user choose an eligible assistant. The app checks Anna's personal classes, other effective duties, reliever duties, event commitments, workload caps, and duty gap before saving. A student with zero duty hours is a valid candidate if those checks pass.

The new Tuesday duty becomes part of the saved working result. Switching to the Student Assistant tab should display it for Anna. Switching back should leave Ben's Monday duty intact. This is one saved assignment set viewed in two ways, not two separate schedules that must be manually copied.

Add Carla and create again without full rescheduling

Carla joins the active file A roster. Leave Reschedule the entire current schedule unchecked. The app identifies assistants not already listed as processed and asks the incremental solver to fit the new student into available duty segments.

Existing valid duties are fixed input. It does not move Ben away from Monday just to give Carla that work. If no usable free segment remains, Carla can finish with zero new duties. That result is compatible with the preserve-existing-work requirement; maximum workload is a ceiling, not a guaranteed allocation.

If Carla already received a manual duty before generation, that fixed work is passed into the incremental context. The solver should adapt to it rather than declare all saved data obsolete simply because the assignment was made manually.

Decide to rebuild everything

If the administrator wants a new distribution, check Reschedule the entire current schedule. This allows regular assignments to change, including manually made duties. Existing events remain separately saved. Replacement records survive only when their original duty still matches the new result.

Review event compatibility afterward. The current full solver does not receive dated event assignments as busy intervals, so saved events alone do not guarantee that regenerated duties avoid them. The manual event-assignment filter and the full solver do not currently have identical inputs.

Add an event and a reliever

Create an orientation event on a concrete date, with a room and time. Assign Anna only if the event-card checks allow her. The event appears on Anna's calendar for that week, not automatically on Ben's or Carla's calendar.

If Ben cannot cover one particular Monday, Find Reliever ranks other eligible students and automatically assigns the first candidate. Assume Carla wins. Ben remains the original recurring assistant. Carla receives a dated reliever record. For the following Monday, Ben is still responsible unless that date also has coverage.

Compare weekly totals

The weekly report adds regular-duty time actually credited that week, assigned event time, and reliever time. Covered duty time is deducted from the original assistant so the same work is not counted for both people as if both performed it. Personal class hours do not count as duty workload.

Different selected weeks can therefore have different totals even though the recurring duty assignments have not changed. Events, exam dates, and replacement dates explain many of these differences.

Save an archive and start the next schedule

Save an archive with a distinct name, school year, and semester. The app captures file A's imported data, relevant roster, duties, events, replacements, settings, and viewing context. After the local save succeeds, the current schedule closes. The snapshot remains accessible through Archives.

Upload file B for another semester. Its different fingerprint selects a different roster and result. Anna need not be included in B merely because she served in A. Viewing A's archive still shows the captured historical roster and duties, regardless of later changes to B.

Sources: Chapters 4 through 18E and their referenced handlers, formatters, solver functions, and storage modules.

23 Scenario reference for everyday actions

The same file is renamed

If the file bytes are unchanged, its SHA-256 fingerprint stays the same. The existing scoped roster and result can be reused even though the display filename changes. Legacy records using a filename fallback can behave differently; they are compatibility data rather than the normal fingerprint path.

A file keeps its name but its content changes

The fingerprint can change. The app treats the new bytes as a different schedule scope. Do not assume matching filenames guarantee matching saved assignments. An Excel program can also change internal workbook bytes during a save even when the visible content appears similar. Archive important work before replacing files.

The active CSV is removed

The normal working timetable becomes empty and the upload action becomes available. The source file on disk is not deleted. Per-file assistant/result caches and scoped events are different from the active selection; they are not all globally erased by closing that selection. Reuploading the same bytes can reconnect to saved scoped data. An archive is the independent historical copy to use when long-term preservation matters.

Only a room is supplied

Recognized room-only input can add that room to the calendar without creating an assignable class. The solver needs a valid work interval and the required class information to create duty work. A room header is a location, not a duty by itself. Arbitrary text in an unrecognized file does not automatically become a room.

Only a time is supplied

Recognized time-only input can contribute a calendar time marker. It does not invent a subject, an assistant assignment, or a valid time range. A marker at 10 AM is different from a complete class scheduled from 10 AM to 12 PM.

A time is invalid or the format is unknown

A supported format can contain an unresolved meeting whose invalid or missing time is reported as TBA rather than a normal assignable interval. A wholly unrecognized header layout is a different error and can reject the upload with Unknown schedule format. TBA handling does not mean the parser accepts every file structure.

An assistant has no duties yet

The profile can exist with zero duty hours. The student can still be considered for a manual duty if available. Absence from the current duty list is not a reason by itself to hide a student from eligible candidates. Availability and the active CSV roster still control whether the student can be chosen.

Create schedule is clicked again with no new assistants

When a result already exists and full rescheduling is unchecked, the app looks for newly unprocessed assistant IDs. If none exist, it reports that situation instead of silently redistributing everyone. Check the full-reschedule option when intentionally requesting a new complete distribution.

There is less work than assistant capacity

Some students can have fewer than the weekly maximum, including zero hours. An upper limit does not create work where none exists. The same class interval cannot be duplicated simply to fill every student's quota. Review actual available slots, eligibility, and coverage rather than treating the maximum as a target that must always be met.

There are too few available assistants

Some work can remain uncovered when the model allows unassigned classes. An uncovered class does not necessarily mean the solver returned INFEASIBLE. It can mean the best found valid assignment set covers only part of the input. Read status, diagnostics, and the assignment list together.

A long duty reaches the gap threshold

Suppose the required gap is 30 minutes. A duty ending at 10 AM after three continuous duty hours prevents another regular duty from starting immediately at 10 AM. A 10:30 AM start can satisfy the gap if other rules also pass. The code evaluates continuous-duty patterns; it does not insert a paid break record into the middle of a class or shorten the department timetable automatically.

Two commitments touch at a boundary

The overlap test uses strict interval intersection. One interval ending at 9 AM and another starting at 9 AM do not overlap solely because they touch. A separate required-gap rule can still make consecutive regular duties invalid. Travel time between rooms is not a separately modeled general rule.

The user edits data while the solver is running

The interface is not globally locked. Before applying a response, the generation handler compares the current CSV key and relevant saved inputs against the request snapshot. A stale answer is discarded rather than overwriting newer work. Discarding a response is not the same as cancelling the backend computation immediately.

The student name changes

The same internal ID should remain, while related display labels are updated. A renamed student should keep compatible duties. The displayed full name, school student ID, and internal record ID serve different purposes. Sorting or display text must not be confused with identity.

The personal schedule changes

The edit planner checks existing commitments against the replacement personal classes. If work becomes incompatible, the UI requires review and confirmation before unassigning affected work. The app rechecks that review context before committing, so an old confirmation is not supposed to remove newer changes silently.

One assistant is removed

Only that student's duties and related replacement links are removed from the working result. Other students' duties remain. The imported department classes remain and can become unassigned. Related event cards remain but lose that student's assignment. Existing immutable archives retain their historical captured data.

A manual duty is removed

The assignment is removed, not the department class. The class can be assigned to another eligible student. The changed working result is persisted with a new result revision. Removing one duty is not a request to rerun the whole solver.

Find Reliever has an eligible candidate

The original assistant is excluded from replacement candidates. Available replacements are ranked, and the first is assigned automatically. The original recurring duty stays saved. Change Reliever becomes relevant after coverage exists; it allows selecting a different eligible replacement instead of regenerating the whole schedule.

No reliever is available

No replacement is invented. The original assistant remains assigned and the UI reports that no reliever is available. This is not evidence that the original assistant is now available to work; it only means the system has not found substitute coverage. The administrator must resolve the operational absence separately.

The administrator wants the original assistant back

Use Remove Reliever to remove the dated coverage record. The original recurring assignment already exists, so it becomes effective again for that date. The original assistant is intentionally not a candidate in the replacement list for their own duty. Returning responsibility is different from selecting the same person as their own substitute.

A reliever exceeds regular workload limits

Reliever work counts as overtime and is not rejected simply for exceeding the regular-duty daily or weekly maximum. Time conflicts still disqualify the student. The recommendation compares workload criteria; it does not prove that the person is suitable under policies not represented in the code.

An event overlaps a personal class or duty

On ordinary dates, the event-assistant filter excludes overlapping personal classes, effective regular work, and replacement duties. Another overlapping assigned event also blocks the choice. Only events assigned to that student should appear in their personal weekly calendar. The normal regular-duty caps are not simply applied as event-overtime caps.

An exam date is selected

Exam records suppress normal classes and duties on that date without deleting the imported timetable. Dated events remain possible. The current event-assignment logic also suspends normal personal-class and duty checks for exam dates, while still rejecting overlapping assigned events. This is an implemented assumption about exam dates, not a guarantee that a student has no real examination then.

An exam draft is cancelled

The temporary calendar preview is removed. Unsaved preview state should not permanently suppress the day's classes. Saved exam markers are different: normal classes return only when no remaining saved exam record keeps that date suppressed.

Rooms are selected together

Select All selects the rooms available to the form. Selecting an occupied room shows the short warning. Existing booking conflicts and occupied-class replacement confirmations can affect whether saving is allowed. A

warning is not an assistant assignment, and selecting every room does not automatically distribute assistants among the resulting cards.

A booking is edited

Matching unchanged cards retain their IDs and assistant links where possible. Unambiguous changed cards can retain identity too, but assistants are revalidated for new times and locations. New cards begin unassigned. Ambiguous changes must not clone one student's assignment into multiple new overlapping cards.

An event is deleted while another device is offline

Migration 010 records the deletion ID and blocks stale writes for that ID. The other device can remove its stale local card after learning the deletion. A deletion made before that protection existed cannot necessarily be reconstructed. Never remove the deletion log merely to make an old queued write succeed.

An archive name already exists

Locally detected duplicates are rejected before saving a second confusing name. If two devices choose the same normalized name while offline, a cloud conflict can appear later. The rejected local snapshot remains available with a conflict state and can be renamed before retry. This exception does not make already-synced snapshots generally editable.

Archive saving fails

A failed local IndexedDB commit must not close the current schedule. If local saving succeeds but the cloud is unreachable, the local snapshot can remain pending for remote sync. A local archive on one computer is not yet an off-device backup until remote saving succeeds.

An archive is viewed or deleted

Viewing uses captured data in read-only mode and does not activate that archive as editable current work. Exiting preview does not mean restoring the archive into the active cache. Deleting an archive records its deletion state; it does not delete the original uploaded file or unrelated active assignments.

Excel is exported

The exporter creates a workbook with separate room worksheets for Monday through Friday of the selected week. Conflict groups are red. Saturday and Sunday are not included by the present exporter even though the app can display them. The workbook is a formatted timetable output, not a full-fidelity archive of all assistant, reliever, revision, and synchronization data.

The app is closed and reopened

Saved local data can be loaded from the same application profile. Normal packaged closing stops the backend process tree owned by that app instance. Separately started development services are different. A forced process kill can bypass normal shutdown. Unsaved form drafts and in-memory state should not be treated as durable records.

Sources: `App.tsx`, `StudentAssistantPanel.tsx`, `ScheduleCalendar.tsx`, `AdminEventsPanel.tsx`, `archiveStorage.ts`, `studentAssistantStorage.ts`, `assistantSchedule.ts`, `weeklyWorkload.ts`, `roomScheduleExcel.ts`, and their regression tests.

24 Checking saved data and diagnosing problems

Separate local availability from remote saving

A visible card proves that the interface currently has that card's data. It does not prove a completed Supabase write. A cloudConfigured health response means usable configuration was found; it does not prove that the project is reachable or that every required migration succeeded. A successful read proves that read worked, not that all later writes will succeed.

When data is absent from Supabase, first check that the correct project and table are open. Then check the app's backend response, required columns and RPCs, cloud availability, and the relevant schedule key. Avoid deleting local storage while diagnosing because it can contain the only copy of a pending change.

Read the tables safely in SQL Editor

These examples only read data. In Supabase SQL Editor, paste a query and click Run. Save stores the query text for reuse; it is not required to commit a normal successful query. Do not approve destructive warnings without understanding the statement that caused them.

List ordinary public tables:

```
select table_name
from information_schema.tables
where table_schema = 'public'
  and table_type = 'BASE TABLE'
order by table_name;
```

Inspect actual columns and types:

```
select table_name, column_name, data_type, is_nullable
from information_schema.columns
where table_schema = 'public'
order by table_name, ordinal_position;
```

Read event details and their CSV ownership:

```
select id, title, event_date, room,
       start_minutes, end_minutes,
       assistant_label, schedule_key
from public.admin_events
order by event_date, room, start_minutes;
```

These timestamps can be inspected too, but the default-now caveat in Section 18B still applies. A NULL schedule_key identifies legacy unscoped data; the filename cannot be recovered reliably from that NULL alone.

Inspect the JSON rather than counting only rows

Read the active timetable's basic identity:

```
select id, csv_name,
       csv_events ->> 'fingerprint' as fingerprint
from public.shared_schedules;
```

Read the scoped roster map and current result:

```
select id,
       solver_result -> 'rostersBySchedule' as rosters,
       solver_result -> 'activeResult' as active_result,
       scheduling_settings
from public.student_assistant_schedules;
```

The arrow operator reads a JSON field. The double arrow returns its text value. Older array or direct-result representations may not contain these envelope fields, so a null result can indicate legacy structure rather than an empty database.

Read archive details without dumping every snapshot:

```
select id, name, school_year, semester,
       created_at, deleted_at
from public.schedule_archives
order by created_at desc;
```

Read known event deletions:

```
select event_id, deleted_at
from public.admin_event_deletions
order by deleted_at desc;
```

Understand errors before trying another save

Response or symptom	Meaning to investigate
HTTP 400	Input validation failed; inspect the returned message
HTTP 409 for archive creation	Normalized archive name conflicts with another active archive
HTTP 410 for an event save	That event ID has already been permanently deleted
HTTP 503	Backend repository or cloud operation failed; inspect the underlying error
Network request fails without a response	Local service, connection, or request delivery problem
Unknown schedule format	Parser did not recognize the actual imported structure
Saved assignments do not match	Existing class identities or intervals cannot be reused as supplied

A 503 is not proof that the CSV is invalid. A parser error is not proof that Supabase is paused. Diagnose the layer producing the error rather than treating every failure as a database problem.

When an old executable behaves differently

The packaged app contains the source as it existed when it was built. Updating a TypeScript or Python file in VS Code does not update an existing executable. Rebuild the package after changes. Confirm that the launched executable is the new output and that an old backend on port 5000 is not serving a different parser.

Port 5173 is the development frontend port. Port 5000 is the local backend port. If npm run dev says 5173 is occupied, find the existing development session and stop it with Ctrl+C in its terminal when appropriate. Do not kill every Python or Node process on the computer.

What one device can and cannot demonstrate

One device can test local persistence by closing and reopening the same app profile, switching between visibly different CSVs, checking immutable archives, and comparing app actions with read-only Supabase queries. Development and packaged profiles may have separate local storage, so they are not automatically one local dataset.

An isolated second app profile with the same backend configuration can help approximate a fresh client. Automated tests can simulate stale saves and failed requests. Neither proves real two-device simultaneous editing under clock differences and network interruptions. Treat those as separate integration tests, not as conclusions from unit tests alone.

Sources: `backend/routes/`, `backend/database/supabase/client.py`, `frontend/src/api/apiClient.ts`, `frontend/src/desktop/main.ts`, migrations 001 through 010, and Chapter 20.

25 System boundaries and a demonstration checklist

What the system currently promises

The application can import recognized timetable formats, retain supported partial room and time input, calculate assistant duties under modeled rules, preserve manual changes, fill remaining work incrementally, generate a full replacement distribution when requested, manage dated events and relievers, calculate weekly workloads, export room timetables, and preserve read-only archives.

Local-first persistence means supported completed actions are saved locally and queued for remote sharing. It does not mean the device cannot run out of storage, every draft is saved, or a cloud request can never fail. An archive saved only locally is still vulnerable to loss of that device's application data.

What is not a hidden feature

The app does not rewrite the original CSV with duty assignments. Archive viewing is not a general Restore to Current operation. The animation is not an actual solver percentage. The shared ccs-main records are not independent per-user workspaces. Normal close cleanup does not guarantee cleanup after a forced crash. An optimal solver status does not prove fairness under policies that were never encoded.

The full solver does not yet consider all dated event commitments as busy intervals. Recommendation ranking does not include event overtime in the same weekly sum used by the report. Exam dates intentionally suppress normal class checks in event selection. Settings are not separately scoped historical configuration records for every CSV. These differences matter when explaining the system accurately.

Security and data responsibility

The current Supabase policies support shared prototype access, not per-user ownership boundaries. Enabling row-level security is insufficient when policies broadly allow access. The packaged environment file can be read from distributed resources; it is not a secure vault. Privileged service-role credentials must not be distributed in a desktop client.

Profiles and schedules can contain student information. Limit access to the project and protect copies of exported workbooks, archives, and local application data. The current code should not be described as a complete authentication, authorization, audit-history, or disaster-recovery solution.

The schema intentionally retains deletion metadata to reject stale writes. This is operational protection, not a complete audit trail of every edit. There is no automatic column-by-column history of all previous names, assignments, and administrator actions.

A practical demonstration sequence

Begin with a clearly labeled test schedule. Upload it and show its rooms and class times. Add two test assistants with visibly different personal-class conflicts. Create a schedule, explain the settings, and point out a duty that would be invalid for one student but allowed for the other.

Manually assign an eligible zero-hour student to an open duty. Switch tabs and show that the assignment persists. Add another student and generate incrementally; demonstrate that previous duties remain. Then explain that full rescheduling is a deliberate different action that permits changes.

Create a dated event, assign an available student, and show that it appears only in that person's selected week. Find a reliever for a regular duty, show the original and replacement states, then remove coverage to restore the original responsibility. Show the weekly workload report and explain why personal-class hours are excluded.

Save an archive and show that current work closes only after local saving. Upload a different schedule with different assistants. View the first archive to prove its historical roster and duties remain independent. Export Excel and explain the current Monday-through-Friday scope and red conflict formatting.

Finally, use read-only Supabase queries to show an event's schedule_key, the assistant roster map, and the archive row. Distinguish a real cloud record from a local-only card. State which checks were actually performed rather than claiming untested simultaneous-device safety.

How to maintain this manual

When behavior changes, update the workflow explanation, relevant database or JSON field descriptions, scenario outcomes, and source reference together. A new column requires a migration; changing only a TypeScript interface does not create that column in Supabase. A new solver preference needs an objective explanation and regression tests; changing a color or button label does not change solver rules.

The source file reference and function index below help locate implementation details. The manual explains each module's role and important algorithms. Tests and the current source remain necessary when investigating an exact input, error, or operating-system condition beyond the worked examples.

Source file reference

The following reference covers application source files, styles, migrations, configuration, and test tooling. Generated builds, dependency packages, spreadsheets, and output artifacts are not independent application features. Empty __init__.py files mark Python packages; they do not contain separate business logic.

The reference includes 138 source, migration, configuration, and verification files. Empty package markers are included for completeness.

Application screens and styling

frontend/src/ui/admin-events/AdminEventsPanel.css (907 lines)

Styles event forms, room availability states, selected dates and times, warnings, and management controls.

frontend/src/ui/admin-events/AdminEventsPanel.tsx (551 lines)

Keeps per-date room and time selections, previews exam dates, validates the booking form, manages existing bookings, and requests confirmations for occupied selections or editing.

frontend/src/ui/admin-events/AutoExpandingTextarea.tsx (20 lines)

Resizes title and description textareas to their content so long text does not remain trapped in a single-line control.

frontend/src/ui/admin-events/bookingForm.ts (54 lines)

Converts stored booking cards into editable form defaults, infers date repetition, and supplies date-label helpers. The saved cards remain the authoritative schedule data.

frontend/src/ui/app/App.css (273 lines)

Styles the application shell, tabs, shared navigation, and top-level layout.

frontend/src/ui/app/App.tsx (501 lines)

Owns the active CSV, fingerprint, scoped admin events, tab selection, archive preview, and cross-screen callbacks. Its upload, event commit, and archive-close handlers connect UI actions to persistence.

frontend/src/ui/app/ThemeToggle.tsx (16 lines)

Displays the accessible light and dark toggle and responds to theme notifications.

frontend/src/ui/app/index.css (79 lines)

Defines global base typography, buttons, form controls, and other shared browser presentation defaults.

frontend/src/ui/app/main.tsx (15 lines)

Applies the saved theme and mounts App in React StrictMode. StrictMode can rerun development effects to expose unsafe side effects; it is not a second solver or database.

frontend/src/ui/app/theme.css (164 lines)

Defines dark and light CSS variables used across feature styles, including stronger light-mode grid and button colors.

frontend/src/ui/app/theme.ts (25 lines)

Applies the chosen theme to the document and notifies mounted controls. Storage access is delegated to themeStorage.ts.

frontend/src/ui/archives/ArchiveManager.tsx (177 lines)

Lists local snapshots first, refreshes shared records separately, saves locally before closing, and offers view, delete, or conflict-only rename actions.

frontend/src/ui/archives/ArchivePreview.tsx (34 lines)

Displays an explicit archive snapshot in read-only mode and switches between its schedule and assistant previews without activating it as current work.

frontend/src/ui/archives/ArchivedAssistantPanel.tsx (55 lines)

Allows choosing an archived assistant, moving through weeks, zooming, and reading the archived workload ranking. It does not run the solver or save edits.

frontend/src/ui/archives/Archives.css (60 lines)

Styles archive dialogs, action rows, archive-list cards, read-only banners, and responsive layouts.

frontend/src/ui/schedule/CalendarZoomControls.css (24 lines)

Styles the small zoom control group used in both timetable screens.

frontend/src/ui/schedule/CalendarZoomControls.tsx (17 lines)

Provides shared minus, reset, and plus controls with 75–200 percent bounds and 25-point steps.

frontend/src/ui/schedule/ScheduleCalendar.css (1081 lines)

Styles the room timetable, absolute-positioned event blocks, conflict colors, responsive controls, dialogs, and focus highlights. Styling changes do not change the solver model.

frontend/src/ui/schedule/ScheduleCalendar.tsx (1170 lines)

Coordinates daily and weekly room views, conflict warnings, uploads, date navigation, filters, duty assignment, event-assistant selection, and relievers. Availability is calculated before saving and relevant actions recalculate it at confirmation.

frontend/src/ui/schedule/ScheduleDateNavigation.css (134 lines)

Keeps the date navigation control consistently sized and styles its picker and navigation buttons.

frontend/src/ui/schedule/ScheduleDateNavigation.tsx (75 lines)

Combines previous and next navigation with the date heading and date-picker interaction. It passes a chosen date to the parent rather than importing a new schedule.

frontend/src/ui/schedule/ScheduleFilter.css (137 lines)

Styles filter choices, dialog layout, and selected states without changing stored schedule records.

frontend/src/ui/schedule/ScheduleFilter.tsx (105 lines)

Maintains room and instructor selections in a filter dialog and passes the chosen sets back to the calendar.

frontend/src/ui/student-assistants/AssistantWeeklyCalendar.tsx (249 lines)

Draws one student’s personal classes, regular duties, assigned events, relieved duties, and replacement duties for a chosen week. It also handles duty details, View and Remove, and read-only preview behavior.

frontend/src/ui/student-assistants/StudentAssistant.css (1091 lines)

Styles the assistant sidebar, profile forms, settings, weekly calendar blocks, duty states, and animated Creating schedule indicator. The indicator is not a measured solver percentage.

frontend/src/ui/student-assistants/StudentAssistantPanel.tsx (652 lines)

Coordinates assistant profiles, parsing personal schedules, editing and deleting students, settings, full or incremental solving, selection, week navigation, and the workload popup.

frontend/src/ui/student-assistants/WeeklyWorkloadSummary.tsx (49 lines)

Renders the roster-wide ranked workload report and its ascending or descending sort control. The calculations are supplied by weeklyWorkload.ts.

API requests storage and formatters

frontend/src/api/adminEventApi.ts (61 lines)

Reads event snapshots with deleted IDs, saves event cards, distinguishes permanent HTTP 410 deletion from temporary failure, and checks protected-delete acknowledgements.

frontend/src/api/apiClient.ts (27 lines)

Builds local backend requests, parses JSON, preserves response status in BackendError, and provides timed polling subscriptions.

frontend/src/api/archiveApi.ts (26 lines)

Transfers archive records and translates duplicate-name responses into ArchiveNameConflictError so storage can preserve the local snapshot and continue other sync work.

frontend/src/api/scheduleApi.ts (33 lines)

Transfers the active imported schedule through the shared schedule endpoint. isCloudConfigured is a compatibility flag here; the backend determines whether credentials are configured.

frontend/src/api/scheduleParser.ts (66 lines)

Reads CSV quoting and XLS/XLSX cell values in the interface, then sends normalized row arrays to the local Python parser.

frontend/src/api/studentAssistantApi.ts (157 lines)

Defines settings, assignments, relievers, solver results, and shared payload contracts. It sends solver requests and verifies backend acknowledgements for scoped and versioned saving.

frontend/src/export/roomScheduleExcel.ts (153 lines)

Creates per-room Monday–Friday XLSX worksheets with exact time boundaries, merged blocks, XML styles, safe names, and red conflict groups, then triggers a browser download.

frontend/src/formatters/archiveName.ts (3 lines)

Normalizes archive names for case-insensitive, repeated-space-insensitive comparisons.

frontend/src/formatters/assistantNameFormatter.ts (11 lines)

Formats available first, middle, and last name fields for full-name display while retaining fallback support for older stored labels.

frontend/src/formatters/assistantSchedule.ts (120 lines)

Plans safe assistant edits and removes one assistant’s assignments without clearing the full result. It updates related labels, checks changed personal schedules, and recalculates totals.

frontend/src/formatters/bookingEdit.ts (70 lines)

Matches existing booking cards to edited cards without duplicating assignments and revalidates assistants for changed slots.

frontend/src/formatters/bookingSeries.ts (3 lines)

Derives a booking-series key by removing the trailing numeric indices and date suffix.

frontend/src/formatters/dateFormatter.ts (36 lines)

Converts local date input values, finds Monday week starts, formats week labels, and matches recurring weekday codes.

frontend/src/formatters/eventSchedule.ts (18 lines)

Filters events by schedule fingerprint and known room names while preserving exam markers and safely hiding unscoped legacy events.

frontend/src/formatters/examPeriod.ts (34 lines)

Recognizes exam marker and booking IDs, encodes schedule ownership, checks dates, and creates internal exam marker records.

frontend/src/formatters/timeFormatter.ts (16 lines)

Converts HH:MM input to minutes and formats numeric minutes and workload durations for people.

frontend/src/formatters/weeklyWorkload.ts (67 lines)

Calculates effective duty, event, and reliever minutes for each student in a selected week, subtracts merged coverage, and sorts the ranking.

frontend/src/storage/adminEventStorage.ts (33 lines)

Stores event cards and permanently known deleted IDs, filters stale records, and emits a local notification when deletions are learned.

frontend/src/storage/archiveStorage.ts (131 lines)

Uses IndexedDB transactions for immutable snapshots, local duplicate-name checks, deletion states, cloud retries, and conflict-only renaming.

frontend/src/storage/archiveTypes.ts (69 lines)

Defines archive data contracts and creates a cloned snapshot scoped to the active CSV, including viewing context.

frontend/src/storage/archiveWorkflow.ts (8 lines)

Waits for a local archive commit before invoking the optional close action. It contains no prerequisite cloud request.

frontend/src/storage/localFirstSync.ts (179 lines)

Persists pending active-schedule, event, and assistant changes; serializes each flush; and acknowledges only the versions actually sent. It is a retry mechanism, not a guarantee of transactional multi-device editing.

frontend/src/storage/preferenceStorage.ts (24 lines)

Persists the selected tab and calendar date and restores valid defaults when saved preference values are absent or invalid.

frontend/src/storage/scheduleStorage.ts (62 lines)

Loads and saves the active imported schedule with compatibility validation for rooms, times, metadata, and TBA fields.

frontend/src/storage/studentAssistantStorage.ts (188 lines)

Stores and activates per-CSV rosters and result caches, builds manual-assignment results, advances actual edit revisions, and merges remote records without replacing newer local data.

frontend/src/storage/themeStorage.ts (11 lines)

Stores the device’s light or dark preference and falls back to dark if storage is unavailable.

Desktop entry points and shared types

frontend/src/desktop/backendProcess.ts (19 lines)

Stops only the spawned backend. On Windows it waits for taskkill to terminate that PID and its descendants; failures are reported rather than silently clearing ownership.

frontend/src/desktop/environment.ts (3 lines)

Reads NODE_ENV to decide whether Electron should use the development URL or packaged resources.

frontend/src/desktop/main.ts (95 lines)

Starts the packaged Python backend, opens the Electron window, chooses development versus packaged content, and coordinates normal shutdown.

frontend/src/desktop/preload.ts (5 lines)

Exposes a small electron bridge containing the local Flask URL. The interface does not receive unrestricted Node access.

frontend/src/desktop/tsconfig.json (19 lines)

Defines TypeScript compilation scope, strictness, module output, and build relationships for its frontend or Electron target.

frontend/src/desktop/tsconfig.preload.json (18 lines)

Defines TypeScript compilation scope, strictness, module output, and build relationships for its frontend or Electron target.

frontend/src/types.ts (99 lines)

Defines CalendarEvent, UploadedAssistant, booking form data, metadata, and other shared TypeScript contracts. Types describe shapes; they do not replace runtime validation of external data.

Python routes models services and solver

backend/app.py (34 lines)

Creates Flask, enables the prototype's CORS configuration, registers all route groups, and converts repository RuntimeErrors to HTTP 503. The main entry point listens on local port 5000.

backend/database/__init__.py (1 lines)

Marks this folder as a Python package. It contains no separate business process.

backend/database/supabase/__init__.py (1 lines)

Marks this folder as a Python package. It contains no separate business process.

backend/database/supabase/admin_event_repository.py (102 lines)

Paginates event and deletion reads, translates application fields to database columns, blocks deleted IDs, and uses the protected delete RPC.

backend/database/supabase/archive_repository.py (45 lines)

Loads complete snapshots and tombstones, creates immutable archives idempotently, detects normalized duplicate names, and marks deletions separately from active schedules.

backend/database/supabase/client.py (82 lines)

Finds the development or frozen environment configuration, checks whether Supabase is configured, performs REST requests with a timeout, and surfaces service errors.

backend/database/supabase/schedule_repository.py (53 lines)

Stores and loads the shared active schedule at ccs-main and supports older list-based versus newer envelope-based JSON storage.

backend/database/supabase/student_assistant_repository.py (116 lines)

Merges per-CSV rosters, unlinked legacy assistants, results, and revisions into the shared assistant cache envelope before saving.

backend/models/__init__.py (1 lines)

Marks this folder as a Python package. It contains no separate business process.

backend/models/admin_event.py (16 lines)

Checks required event fields, nonblank name and room, schedule key when supplied, and an end time after the start.

backend/models/archive.py (37 lines)

Validates snapshot version, ID consistency, required details, timestamp, schedule lists, assistant data, and viewing dates.

backend/models/schedule.py (48 lines)

Validates the shared schedule object and its list fields before storage.

backend/models/student_assistant.py (13 lines)

Checks that shared or solver input is a JSON object. Detailed scheduling validation occurs inside the solver and repositories; this is not a complete nested schema validator.

backend/requirements.txt (4 lines)

Declares Flask, CORS support, OR-Tools, and PyInstaller dependencies for the Python side.

backend/routes/__init__.py (1 lines)

Marks this folder as a Python package. It contains no separate business process.

backend/routes/admin_events.py (38 lines)

Exposes ordinary event reads, deletion-aware sync snapshots, event PUT, and protected DELETE; distinguishes a deleted event with HTTP 410.

backend/routes/archives.py (31 lines)

Exposes archive listing, immutable creation, and deletion; distinguishes a duplicate name with HTTP 409.

backend/routes/health.py (10 lines)

Returns an OK status and whether the local backend has usable cloud configuration; it does not prove the cloud project is reachable.

backend/routes/schedules.py (37 lines)

Receives parser requests and shared active-schedule GET and PUT requests; returns validation errors as HTTP 400.

backend/routes/student_assistants.py (33 lines)

Receives shared assistant data and solver requests; returns acknowledgement versions needed by local retry queues.

backend/services/__init__.py (1 lines)

Marks this folder as a Python package. It contains no separate business process.

backend/services/admin_event_service.py (24 lines)

Validates event updates, delegates protected deletion, and builds sync snapshots filtered against deleted IDs.

backend/services/archive_service.py (21 lines)

Validates snapshot payloads and UUIDs before invoking archive creation, listing, or deletion.

backend/services/conflict_detector.py (9 lines)

Provides a small room-and-interval overlap predicate. It does not itself compare recurring weekdays or dates and is not the live calendar's main conflict implementation.

backend/services/schedule_parser.py (600 lines)

Contains legacy, official-template, personal-schedule, and course-offering parsers. It normalizes headers, times, weekdays, sections, credits, metadata, TBA entries, room-only rows, and time-only rows.

backend/services/schedule_service.py (12 lines)

Validates the shared schedule request and delegates loading or saving to the schedule repository.

backend/services/student_assistant_service.py (17 lines)

Connects assistant storage requests to validation and repository calls and solver requests to OR-Tools.

backend/services/vacant_room_finder.py (12 lines)

Uses the simple backend overlap helper to return rooms without a conflicting supplied record. No current Flask route calls this helper; live room-picking logic is in AdminEventsPanel.

backend/solver/__init__.py (1 lines)

Marks this folder as a Python package. It contains no separate business process.

backend/solver/student_assistant_solver.py (758 lines)

Builds OR-Tools CP-SAT decision variables, hard constraints, and weighted preferences; supports incremental filling; and returns assignments, totals, diagnostics, and applied settings.

Database migrations

database/supabase/migrations/001_shared_schedule.sql (52 lines)

Creates the shared active-schedule table and prototype access policies.

database/supabase/migrations/002_admin_events.sql (84 lines)

Creates individual event rows, imports legacy events, and defines event access and realtime publication setup.

database/supabase/migrations/003_student_assistant_schedules.sql (51 lines)

Creates shared assistant-profile and solver-result storage.

database/supabase/migrations/004_admin_event_assistants.sql (4 lines)

Adds assistant identity and display-label fields to event rows.

database/supabase/migrations/005_duty_break_setting.sql (14 lines)

Adds JSON scheduling settings storage used for duty-gap and workload values.

database/supabase/migrations/006_admin_event_descriptions.sql (3 lines)

Adds event descriptions without changing the imported class format.

database/supabase/migrations/007_schedule_archives.sql (42 lines)

Creates the separate archive table, immutable-snapshot guard, and shared prototype policies.

database/supabase/migrations/008_csv_scoped_events.sql (7 lines)

Adds the schedule key and supporting index that associate each event with a specific uploaded schedule.

database/supabase/migrations/009_unique_archive_names.sql (6 lines)

Enforces unique normalized names among nondeleted archives. It does not automatically rename conflicting existing snapshots.

database/supabase/migrations/010_admin_event_deletions.sql (60 lines)

Creates read-only deletion records, a direct-write trigger guard, and an atomic delete RPC. The retained IDs prevent offline event resurrection.

Build configuration and verification scripts

backend/tests/test_admin_event_deletions.py (67 lines)

Checks protected deletes, missing migrations, rejected stale saves, route statuses, pagination, and the migration guard structure.

backend/tests/test_admin_event_descriptions.py (45 lines)

Checks description persistence and that missing required schema support is not silently acknowledged.

backend/tests/test_archives.py (102 lines)

Checks archive schema, idempotent creation, duplicate names, immutable payload storage, deletion states, and route responses.

backend/tests/test_assistant_rosters.py (124 lines)

Checks scoped roster merging and result revisions, including stale inactive caches, offline multi-CSV changes, clears, and older clients.

backend/tests/test_event_schedule.py (36 lines)

Checks event schedule keys, assistant persistence, and rejection of invalid scope values.

backend/tests/test_incremental_solver.py (179 lines)

Checks preserved old duties, newly added students, remaining slots, and incremental validation rules.

backend/tests/test_schedule_parser.py (102 lines)

Checks format detection, shared meetings, metadata, invalid-time TBA behavior, complete official templates, and persisted parsed data.

backend/tests/test_student_assistant_service.py (45 lines)

Exercises solver rules and result handling using small deterministic fixtures.

eslint.config.js (25 lines)

Configures static code-quality checks for the project.

frontend/electron-builder.json (21 lines)

Defines the app ID, bundled frontend and desktop files, backend resources, copied environment file, icon, and portable/MSI targets.

frontend/index.html (13 lines)

Provides the root HTML page and root element into which React mounts the interface.

frontend/tsconfig.app.json (25 lines)

Defines TypeScript compilation scope, strictness, module output, and build relationships for its frontend or Electron target.

frontend/tsconfig.json (7 lines)

Defines TypeScript compilation scope, strictness, module output, and build relationships for its frontend or Electron target.

frontend/tsconfig.node.json (25 lines)

Defines TypeScript compilation scope, strictness, module output, and build relationships for its frontend or Electron target.

frontend/vite.config.ts (18 lines)

Sets the frontend root, relative packaged asset paths, build output, and development server port 5173.

package.json (53 lines)

Lists application dependencies and development, build, test, and platform-packaging commands.

scripts/build-backend.mjs (52 lines)

Locates Python and runs PyInstaller with the backend import path and OR-Tools collection. Its current onefile output is packaged as an Electron resource.

scripts/register-typescript.mjs (28 lines)

Registers a Node loader for testing actual TS and TSX modules without writing a separate test-only implementation.

scripts/setup-python.mjs (36 lines)

Installs backend requirements during npm installation and reports missing Python or package-installation failures.

scripts/test-archives.mjs (209 lines)

Runs regression checks for archives, using the actual related modules and test fixtures.

scripts/test-assignment-sync.mjs (69 lines)

Runs regression checks for assignment sync, using the actual related modules and test fixtures.

scripts/test-assistant-edit.mjs (42 lines)

Runs regression checks for assistant edit, using the actual related modules and test fixtures.

scripts/test-assistant-removal.mjs (55 lines)

Runs regression checks for assistant removal, using the actual related modules and test fixtures.

scripts/test-assistant-rosters.mjs (99 lines)

Runs regression checks for assistant rosters, using the actual related modules and test fixtures.

scripts/test-backend-process.mjs (51 lines)

Tests shutdown commands and failure handling; its optional Windows integration mode launches and terminates only its own disposable process tree.

scripts/test-booking-edit.mjs (42 lines)

Runs regression checks for booking edit, using the actual related modules and test fixtures.

scripts/test-conflicts-database.mjs (122 lines)

Runs regression checks for conflicts database, using the actual related modules and test fixtures. This separate check may contact a configured service; inspect its fixture and configuration before running it.

scripts/test-conflicts.mjs (86 lines)

Runs regression checks for conflicts, using the actual related modules and test fixtures.

scripts/test-event-deletions.mjs (88 lines)

Runs regression checks for event deletions, using the actual related modules and test fixtures.

scripts/test-event-schedule.mjs (56 lines)

Runs regression checks for event schedule, using the actual related modules and test fixtures.

scripts/test-exam-period.mjs (109 lines)

Runs regression checks for exam period, using the actual related modules and test fixtures.

scripts/test-preferences.mjs (25 lines)

Runs regression checks for preferences, using the actual related modules and test fixtures.

scripts/test-room-schedule-export.mjs (49 lines)

Runs regression checks for room schedule export, using the actual related modules and test fixtures.

scripts/test-schedule-import.mjs (87 lines)

Runs regression checks for schedule import, using the actual related modules and test fixtures.

scripts/test-theme.mjs (64 lines)

Runs regression checks for theme, using the actual related modules and test fixtures.

scripts/test-ui-components.mjs (81 lines)

Runs regression checks for ui components, using the actual related modules and test fixtures.

scripts/test-weekly-workload.mjs (24 lines)

Runs regression checks for weekly workload, using the actual related modules and test fixtures.

scripts/theme-ui-smoke.cjs (305 lines)

Runs isolated hidden Electron UI tests with synthetic local storage and mocked HTTP calls. Optional modes cover assistant edits/removal, booking edits, event deletion, and archive save/rename.

Important function and handler index

This index identifies named functions, exported data contracts, and selected screen declarations in the current source. Line numbers are navigation aids for this snapshot and will change when files are edited. For the reasoning behind the functions, use the preceding process chapters rather than treating a function name as a complete explanation.

backend/app.py

create_app at line 13

backend/database/supabase/admin_event_repository.py

EventDeletedError at line 8; load_deleted_event_ids at line 12; _load_event_rows at line 24; load_events at line 40; save_event at line 74; delete_event at line 101

backend/database/supabase/archive_repository.py

ArchiveNameConflictError at line 7; load_archives at line 11; create_archive at line 23; delete_archive at line 42

backend/database/supabase/client.py

_load_project_environment at line 14; is_configured at line 38; request at line 47

backend/database/supabase/schedule_repository.py

load_schedule at line 8; save_schedule at line 42

backend/database/supabase/student_assistant_repository.py

load_data at line 8; save_data at line 51

backend/models/admin_event.py

validate_admin_event at line 4

backend/models/archive.py

validate_archive at line 6

backend/models/schedule.py

validate_shared_schedule at line 4

backend/models/student_assistant.py

validate_solver_request at line 4; validate_shared_assistant_data at line 10

backend/routes/admin_events.py

get_admin_events at line 15; put_admin_event at line 22; remove_admin_event at line 36

backend/routes/archives.py

list_archives at line 10; put_archive at line 15; delete_archive at line 26

backend/routes/health.py

health at line 9

backend/routes/schedules.py

parse_schedule at line 13; get_shared_schedule at line 24; put_shared_schedule at line 29

backend/routes/student_assistants.py

get_shared_data at line 13; put_shared_data at line 18; solve_student_assistants at line 28

backend/services/admin_event_service.py

get_admin_events at line 7; get_admin_event_sync at line 11; update_admin_event at line 19; remove_admin_event at line 23

backend/services/archive_service.py

get_archives at line 8; save_archive at line 12; remove_archive at line 16

backend/services/conflict_detector.py

events_overlap at line 4

backend/services/schedule_parser.py

clean at line 23; header_key at line 27; parse_time at line 31; looks_like_class_clock at line 49; normalize_day at line 61; field_for_header at line 72; detect_header at line 77; infer_columns at line 91; _parse_legacy_schedule_rows at line 169; extract_student_id at line 251; _parse_assistant_schedule_rows at line 273; _parse_official_schedule_rows at line 328; offering_header at line 461; offering_time at line 473; _parse_course_offering_rows at line 490; parse_schedule_rows at line 581

backend/services/schedule_service.py

get_shared_schedule at line 7; update_shared_schedule at line 11

backend/services/student_assistant_service.py

get_student_assistant_data at line 8; save_student_assistant_data at line 12; create_student_assistant_schedule at line 16

backend/services/vacant_room_finder.py

find_vacant_rooms at line 6

backend/solver/student_assistant_solver.py

Meeting at line 22; CoverageUnit at line 35; _expand_days at line 46; _parse_meetings at line 54; _coverage_units at line 80; _overlaps at line 100; _duty_gap_minutes at line 108; _workload_limits at line 124; _add_duty_break_constraints at line 148; _merge_assignments at line 218; _assignment_payload at line 272; _solve_incremental_schedule at line 293; solve_student_assistant_schedule at line 449

frontend/src/api/adminEventApi.ts

SharedAdminEvent at line 4; loadSharedAdminEvents at line 22; saveSharedAdminEvent at line 29; deleteSharedAdminEvent at line 53; subscribeToSharedAdminEvents at line 59

frontend/src/api/apiClient.ts

BackendError at line 3; requestJson at line 8; pollForChanges at line 24

frontend/src/api/archiveApi.ts

loadSharedArchives at line 4; ArchiveNameConflictError at line 8; createSharedArchive at line 10; deleteSharedArchive at line 24

frontend/src/api/scheduleApi.ts

SharedSchedule at line 4; loadSharedSchedule at line 18; saveSharedSchedule at line 23; subscribeToSharedSchedule at line 31

frontend/src/api/scheduleParser.ts

parseCsvRows at line 5; rowsFromFile at line 34; readScheduleFile at line 58

frontend/src/api/studentAssistantApi.ts

StudentAssistantInput at line 4; SchedulingSettings at line 10; normalizeSchedulingSettings at line 26; DutyAssignment at line 45; AssistantTotal at line 58; RelieverAssignment at line 65; StudentAssistantResult at line 79; solveStudentAssistantSchedule at line 99; SharedStudentAssistantData at line 119; loadSharedStudentAssistantData at line 131; saveSharedStudentAssistantData at line 139; subscribeToSharedStudentAssistantData at line 155

frontend/src/desktop/backendProcess.ts

stopBackendProcess at line 5

frontend/src/desktop/environment.ts

isDev at line 1

frontend/src/desktop/main.ts

startFlask at line 13; stopFlask at line 49

frontend/src/export/roomScheduleExcel.ts

escapeXml at line 12; timeLabel at line 18; eventText at line 22; cell at line 28; sheetNames at line 32; createRoomScheduleExcel at line 47; valid at line 62; downloadRoomScheduleExcel at line 142

frontend/src/formatters/archiveName.ts

normalizedArchiveName at line 1

frontend/src/formatters/assistantNameFormatter.ts

assistantDisplayName at line 3

frontend/src/formatters/assistantSchedule.ts

AssistantEditConflict at line 7; AssistantEditPlan at line 13; planAssistantEdit at line 20; overlapsClass at line 30; label at line 37; dutyKey at line 40; assignments at line 42; relievers at line 49; withoutAssistantAssignments at line 85; assignments at line 93

frontend/src/formatters/bookingEdit.ts

editedBookingEvents at line 9; validateEditedEventAssistants at line 46

frontend/src/formatters/bookingSeries.ts

bookingSeriesKey at line 1

frontend/src/formatters/dateFormatter.ts

toDateInputValue at line 1; startOfWeek at line 8; dateFromInputValue at line 16; weekRangeLabel at line 21; matchesSelectedDay at line 32

frontend/src/formatters/eventSchedule.ts

eventBelongsToSchedule at line 5; eventsForSchedule at line 10

frontend/src/formatters/examPeriod.ts

isExamPeriod at line 5; isExamBooking at line 9; examPeriodPrefix at line 13; examBelongsToSchedule at line 17; isExamDate at line 22; createExamPeriodEvents at line 26

frontend/src/formatters/timeFormatter.ts

parseInputTime at line 1; formatTime at line 6; workloadHoursLabel at line 13

frontend/src/formatters/weeklyWorkload.ts

WeeklyWorkload at line 6; weeklyWorkloads at line 15; sortWeeklyWorkloads at line 64

frontend/src/storage/adminEventStorage.ts

loadDeletedEventIds at line 7; rememberDeletedEvents at line 11; loadAdminEvents at line 20; saveAdminEventsLocally at line 30

frontend/src/storage/archiveStorage.ts

openDatabase at line 10; transaction at line 20; listLocalArchives at line 33; saveArchiveLocally at line 40; deleteArchiveLocally at line 53; renameConflictingArchive at line 65; mergeArchiveRecord at line 85; acknowledge at line 92; syncArchives at line 105

frontend/src/storage/archiveTypes.ts

ArchiveView at line 5; ScheduleArchive at line 12; ArchiveRecord at line 25; RemoteArchiveRecord at line 33; createScheduleArchive at line 39

frontend/src/storage/archiveWorkflow.ts

saveArchiveWithAction at line 5

frontend/src/storage/localFirstSync.ts

AdminQueue at line 16; AssistantSnapshot at line 21; readJson at line 37; sameValue at line 46; hasPendingScheduleSync at line 50; queueScheduleSync at line 54; flushPendingScheduleSync at line 59; loadAdminQueue at line 71; saveAdminQueue at line 79; queueAdminUpsert at line 87; queueAdminChanges at line 92; queueAdminDelete at line 109; mergePendingAdminEvents at line 113; flushPendingAdminSync at line 125; hasPendingAssistantSync at line 150; queueAssistantSync at line 154; flushPendingAssistantSync at line 159

frontend/src/storage/preferenceStorage.ts

loadActiveTab at line 6; saveActiveTab at line 11; saveScheduleDate at line 15; loadScheduleDate at line 19

frontend/src/storage/scheduleStorage.ts

SavedSchedule at line 3; validTbaSubjects at line 15; validRooms at line 24; validTimes at line 35; loadCsvSchedule at line 40; saveCsvScheduleLocally at line 60

frontend/src/storage/studentAssistantStorage.ts

LocalAssistantData at line 14; withManualDutyAssignments at line 26; loadLocalAssistantData at line 71; saveLocalAssistantData at line 109; activateLocalAssistantSchedule at line 139; mergeSharedAssistantData at line 155

frontend/src/storage/themeStorage.ts

Theme at line 1; loadTheme at line 4; saveTheme at line 9

frontend/src/types.ts

Tab at line 1; EventSource at line 2; CalendarEvent at line 4; ScheduleMetadata at line 30; ScheduleImportResult at line 35; ScheduleConflict at line 44; AdminEventForm at line 51; BookingRepeat at line 60; BookingEditScope at line 61; RoomBookingForm at line 63; BookingDateSchedule at line 76; UploadedAssistant at line 82

frontend/src/ui/admin-events/AdminEventsPanel.tsx

EventTimeRange at line 12; AdminEventsPanel at line 14; updateBookingTime at line 205; addBookingTime at line 211; removeBookingTime at line 219; resetDateSelectionToToday at line 227; toggleBookingDate at line 240; selectDatesInBetween at line 276; openDatePicker at line 293; applyCalendarDates at line 301; applyTimesToDates at line 308; applyRoomsToDates at line 314; submitBooking at line 322; completeBooking at line 347; selectBookingRoom at line 373; selectAllRooms at line 383; clearActiveRooms at line 389; completeMove at line 395; submitMove at line 419; openApplyDatesDropdown at line 460; cancelApplyDatesDropdown at line 464; applySelectedDatesAndClose at line 468; renderApplyDatesDropdown at line 474

frontend/src/ui/admin-events/AutoExpandingTextarea.tsx

AutoExpandingTextarea at line 4

frontend/src/ui/admin-events/bookingForm.ts

defaultSelectableDate at line 5; createEmptyBookingForm at line 11; inferBookingRepeat at line 18; formatSelectedDate at line 36; bookingFormFromEvents at line 40

frontend/src/ui/app/App.tsx

filenameScheduleKey at line 38; App at line 43; uploadCsv at line 209; removeCsv at line 229; commitEventEdit at line 243; saveAdminEvent at line 271; saveRoomBooking at line 301; deleteAdminEvent at line 330; assignEventAssistant at line 340; deleteAdminEvents at line 353; updateRoomBooking at line 364

frontend/src/ui/app/ThemeToggle.tsx

ThemeToggle at line 4

frontend/src/ui/app/theme.ts

currentTheme at line 5; applyTheme at line 9; subscribeTheme at line 15; onStorage at line 16

frontend/src/ui/archives/ArchiveManager.tsx

ArchiveSummary at line 8; ArchiveManagerProps at line 15; ArchiveManager at line 27; refreshLocal at line 69; save at line 70; rename at line 87; confirmDelete at line 100

frontend/src/ui/archives/ArchivePreview.tsx

ArchivePreview at line 9

frontend/src/ui/archives/ArchivedAssistantPanel.tsx

ArchivedAssistantPanel at line 11; assignments at line 23; relievers at line 26; moveWeek at line 31

frontend/src/ui/schedule/CalendarZoomControls.tsx

CalendarZoomControlsProps at line 3; CalendarZoomControls at line 9

frontend/src/ui/schedule/ScheduleCalendar.tsx

DutyContext at line 22; RelieverCandidate at line 28; scheduleIdentifier at line 36; conflictLabel at line 41; calendarEventKey at line 47; sameWeek at line 51; overlaps at line 55; longestConsecutiveMinutes at line 59; violatesDutyGap at line 77; teacherKey at line 94; abbreviatedAssistantName at line 101; ScheduleCalendar at line 108; assignmentForEvent at line 216; relieverForEvent at line 219; assistantForEvent at line 229; openAssistantAssignment at line 295; saveEventAssistant at line 300; removeEventAssistant at line 310; openEventEditor at line 318; positionForMinute at line 548; openFilters at line 563; applyFilters at line 567; moveWeeklyRoom at line 573; navigateToConflict at line 580; handleUpload at line 591; closeRelieverFlow at line 604; openAbsenceFlow at line 611; effectiveIntervalsFor at line 622; effectiveWeeklyMinutes at line 650; rankRelievers at line 672; regularDutyIntervalsFor at line 723; regularWeeklyMinutesFor at line 738; availableRegularDutyCandidates at line 749; openUnassignedDuty at line 788; assignRegularDuty at line 794; relieverRecordFor at line 823; persistRelieverRecords at line 837; findRelievers at line 853; changeReliever at line 869; saveChangedReliever at line 875; removeReliever at line 883; instructorName at line 1157

frontend/src/ui/schedule/ScheduleDateNavigation.tsx

ScheduleDateNavigationProps at line 7; ScheduleDateNavigation at line 13; move at line 43; select at line 50

frontend/src/ui/schedule/ScheduleFilter.tsx

ScheduleFilterOption at line 4; ScheduleFilter at line 9; toggleOption at line 41; clearFilters at line 52

frontend/src/ui/student-assistants/AssistantWeeklyCalendar.tsx

PreviewDetails at line 12; expandedDayCodes at line 31; AssistantWeeklyCalendar at line 37; previewProps at line 63; position at line 106

frontend/src/ui/student-assistants/StudentAssistantPanel.tsx

storedAssistantName at line 33; dutyGapLabel at line 38; StudentAssistantPanel at line 45; saveAssistantData at line 154; openSchedulingSettings at line 173; proceedWithSchedule at line 182; closeAddProfile at line 202; closeEditProfile at line 209; openEditProfile at line 220; addStudentAssistant at line 250; saveEditedAssistant at line 296; deleteStudentAssistant at line 377; removeDutyAssignment at line 397; runSolver at line 423; moveViewedWeek at line 537; visibleDiagnostics at line 556

frontend/src/ui/student-assistants/WeeklyWorkloadSummary.tsx

WeeklyWorkloadSummaryProps at line 5; WeeklyWorkloadSummary at line 13

Glossary

Term	Meaning in this application
Backend	The local Python service that handles requests and runs the solver
Boolean decision variable	A solver value that is either selected or not selected
Constraint	A condition that a valid solver answer must satisfy
Fingerprint	A SHA-256 identifier derived from uploaded file bytes
Hydration	Loading saved local or remote data into the current application's state
Incremental solve	Filling available duties for newly added assistants while preserving saved assignments
IndexedDB transaction	A local database operation that commits or fails as a unit
JSON envelope	An object holding several related fields or keyed caches inside one stored value
Objective	The weighted preferences the solver tries to maximize among valid choices
Optimized assistant IDs	The saved set used to distinguish previously processed assistants from new additions
Pending queue	Locally stored changes awaiting successful synchronization
Read only	Viewing captured data with editing actions disabled
Revision	A per-CSV marker used to compare newer and older saved changes
Roster	The assistants associated with one schedule fingerprint
Snapshot	A captured copy of related data at a particular time
Tombstone	A retained deletion record that prevents stale data from being recreated
Upsert	Insert a new record or update the existing record with the same key